

Data Engineering Basics for ML

An interview-focused guide to the storage, transformation, orchestration, and distributed-computing foundations behind production machine-learning systems. Examples use Python, SQL, PySpark, Airflow, and Kafka where appropriate. Complexity symbols used throughout: **N** rows, **M** columns, **P** partitions, and **K** distinct grouping or join keys.

CSV, JSON, and Parquet

1. Overview

CSV, JSON, and Parquet are common ways to exchange or persist ML data. CSV is a flat, delimiter-separated text format; JSON represents nested records and arrays; Parquet is a typed, compressed, columnar binary format. CSV is useful for small tabular exports, JSON for APIs and event payloads, and Parquet for analytical scans, feature stores, data lakes, and large training datasets. Choosing the wrong format can multiply storage, parsing time, and cloud scan cost.

2. Intuition

Think of CSV as a printed spreadsheet, JSON as a labeled folder tree, and Parquet as a warehouse whose shelves are organized by column. If an ML job needs only **age** and **label** from 200 columns, Parquet can read those shelves only; CSV/JSON normally parse every row and field.

3. Prerequisites

- Rows, columns, schemas, primitive types, nulls, and character encodings.
- Python file I/O and pandas basics.
- Compression, serialization, partitions, and train/validation/test data separation.

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
CSV schema	Types are not stored reliably; readers infer or receive a schema.	"001" may become integer 1.	Why can inference corrupt IDs?
CSV quoting	Delimiters/newlines inside values require quoting and escaping.	"Delhi, India",42	Why is splitting on commas incorrect?
JSON nesting	Objects and arrays preserve hierarchy but complicate analytics.	{"user": {"id":7},"items": [...]}	JSON vs JSON Lines?

Subtopic	Meaning and importance	Example	Interview angle
JSONL/NDJSON	One JSON object per line supports streaming and split reads.	One event per line.	Why is a giant JSON array hard to parallelize?
Parquet columnar layout	Values from each column are stored together in row groups.	Read only two of 200 features.	Explain column pruning.
Statistics and predicate pushdown	Min/max/null-count metadata lets engines skip row groups.	Skip groups where <code>date < 2026-01-01</code> .	File pruning vs row filtering?
Compression/encoding	Repeated typed values compress efficiently using dictionary, RLE, bit packing.	Country codes dictionary-encoded.	Why does Parquet usually beat gzip CSV?
Schema evolution	Additive nullable fields are safer than renaming or changing types.	Add <code>device_type STRING</code> .	How do readers handle old files?

5. Algorithm / Working Process

1. A writer maps in-memory values to a logical schema.
2. CSV writes escaped textual fields; JSON writes keys and nested values; Parquet batches rows into row groups and column chunks.
3. Compression is applied at file or column-page level.
4. On read, the engine discovers or receives a schema, selects files/partitions, and parses/decompresses data.
5. Parquet readers use footer metadata for projection and predicate pushdown before materializing rows.
6. The resulting table becomes validation, feature engineering, or model-training input.

6. Mathematical Foundation

Compression ratio is

$$R_c = \frac{\text{uncompressed bytes}}{\text{compressed bytes}}.$$

If a row-oriented file has M equal-width columns and a query reads m columns, ideal projected I/O is approximately

$$\text{I/O}_{\text{columnar}} \approx \frac{m}{M} \text{I/O}_{\text{row}}, \quad m \ll M.$$

Actual I/O includes metadata, pages, and row-group boundaries. Predicate pushdown skips row group g for $x > t$ if metadata proves $\max(x_g) \leq t$. Format choice has no loss function; evaluate correctness, bytes stored/scanned, read/write throughput, and schema stability.

7. Practical Implementation

```

from pathlib import Path
import json
import pandas as pd

rows = [
    {"user_id": "001", "age": 24, "country": "IN", "label": 1},
    {"user_id": "002", "age": None, "country": "US", "label": 0},
]
df = pd.DataFrame(rows)

# Preserve identifier semantics explicitly.
df.to_csv("users.csv", index=False)
Path("users.jsonl").write_text(
    "".join(json.dumps(row) + "\n" for row in rows), encoding="utf-8"
)
df.to_parquet("users.parquet", index=False) # requires pyarrow or fastparquet

csv_df = pd.read_csv("users.csv", dtype={"user_id": "string"})
json_df = pd.read_json("users.jsonl", lines=True)
parquet_df = pd.read_parquet("users.parquet", columns=["age", "label"])

assert csv_df["user_id"].iloc[0] == "001"
assert list(parquet_df.columns) == ["age", "label"]

```

8. Code Explanation

`dtype` prevents pandas from treating a zero-padded identifier as a number. JSON Lines allows record-by-record processing. `columns=` demonstrates Parquet projection: unrelated columns need not be decoded. In production, also specify a schema at ingestion and avoid relying on inference from a small sample.

9. Training / Evaluation

These are data formats, not trained models. Evaluate them with a representative dataset and workload: verify schema and row counts; benchmark cold-cache and warm-cache reads; measure bytes stored and scanned; confirm null, timestamp, decimal, and Unicode round trips; and ensure train/validation/test files cannot overlap. Parquet row-group size, compression codec, file size, and partition columns act like operational hyperparameters.

10. Complexity and Cost

- CSV/JSON parsing is typically $O(NM)$ and CPU-heavy because text must be converted to types.
- A projected Parquet scan is approximately $O(Nm)$ for selected columns, with further savings from skipped row groups.
- CSV is often largest; JSON repeats keys; compressed Parquet is usually smallest for analytic tables.
- Too many tiny Parquet files cause metadata and scheduler overhead; huge files reduce parallelism.
- GPU is unnecessary for format conversion; CPU, memory, disk bandwidth, and object-store requests dominate.

11. Common Use Cases

- CSV: manual exchange, public datasets, debugging samples, spreadsheet interoperability.
- JSON/JSONL: REST APIs, logs, semi-structured events, model request/response records.
- Parquet: offline feature tables, lakehouse tables, large training corpora metadata, BI analytics.

12. Common Mistakes

- Inferring types and losing leading zeros, decimal precision, or timezone information.
- Parsing CSV with `split(',')`; ignoring quotes, escapes, encoding, and embedded newlines.
- Storing one huge JSON array instead of splittable JSONL.
- Partitioning Parquet by high-cardinality IDs and creating millions of directories/files.
- Mixing incompatible schemas or different units in one dataset.
- Fitting preprocessing before the train/test split; the format does not prevent leakage.

13. Edge Cases / Limitations

CSV cannot naturally represent nested values or distinguish an empty string from null without a convention. JSON numbers may lose large-integer precision in JavaScript consumers. Parquet is not human-readable and is inefficient for frequent single-row updates; use a table format or database for transactional semantics. Corrupt files, mixed encodings, duplicate keys, NaN/Infinity, and daylight-saving timestamps require explicit policies.

14. Variations

- Avro: row-oriented binary format with strong schema evolution; important for Kafka and data exchange.
- ORC: columnar format common in Hive ecosystems; similar interview relevance to Parquet.
- Arrow IPC/Feather: columnar in-memory interchange optimized for fast local transfer.
- Delta/Iceberg/Hudi: table layers over Parquet adding transactions, snapshots, and evolution; important for projects and senior interviews.

15. Related Topics

Parquet connects to columnar warehouses, Spark predicate pushdown, and lakehouse table formats. JSON events often enter Kafka, become normalized by ETL, and land as partitioned Parquet. Data-quality checks enforce the schema and semantic constraints that raw formats alone cannot guarantee.

16. Interview Questions

1. **CSV vs Parquet?** CSV is portable text without robust types; Parquet is typed, compressed, and columnar, making analytical projections faster.
2. **Why is Parquet good for ML training tables?** It preserves types and supports parallel, projected reads with efficient compression.
3. **What is predicate pushdown?** Applying a filter at the storage reader so metadata can skip blocks before decoding rows.
4. **What is column pruning?** Reading only referenced columns.
5. **JSON vs JSONL?** JSON may be one document; JSONL stores independent records, enabling streaming and easier splitting.
6. **Why are many small files harmful?** File listing, open requests, footer reads, and task scheduling dominate useful computation.

7. **How would you store images plus labels?** Usually object-store image files with a Parquet manifest; large sharded record formats are another option.
8. **Can Parquet enforce uniqueness?** No; a processing or table layer must enforce/check it.
9. **How do you handle schema evolution?** Prefer additive nullable fields, stable field IDs/names, compatibility checks, and versioned contracts.
10. **Why explicitly define schemas?** Inference can vary by sample and silently alter IDs, nullability, timestamps, or numeric precision.

17. Practice Tasks

- Code: convert messy CSV to typed Parquet while logging rejected rows.
- Dataset project: benchmark selected-column reads on a public taxi dataset.
- Experiment: compare Snappy, gzip, and Zstandard storage and read latency.
- Debugging: find why a zero-padded ID and timezone changed after a round trip.
- Extension: partition by date and demonstrate partition plus row-group pruning.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Format benchmark lab	Compares size, throughput, and projection	pandas, PyArrow; NYC Taxi	Shows evidence-based storage decisions
Schema-safe ingestion	Validates CSV/JSON and writes quarantined errors plus Parquet	Python, Pandera/Pydantic	Demonstrates robust ingestion
Image manifest builder	Creates split-safe metadata for vision training	Python, Parquet; COCO	Connects storage design to ML loaders

19. Quick Revision

- Key idea: match format to access pattern; Parquet for analytics, JSON for nested exchange, CSV for simple interoperability.
- Main formula: projected columnar I/O is roughly (m/M) of a full row scan.
- Metrics: size, read/write throughput, scanned bytes, parse failures.
- Trap: inferred schema and tiny files.
- Interview one-liner: "Parquet wins analytical workloads through typed columnar storage, compression, projection, and row-group skipping."

20. Final Cheat Sheet

Item	CSV	JSON/JSONL	Parquet
Input/output	Flat rows/text	Nested records/text	Typed tables/binary
Main steps	Quote, delimit, parse	Serialize, parse, normalize	Encode columns, compress, read footer
Key tuning	delimiter, encoding	JSONL, schema	codec, row-group/file size, partitions
Pros	Universal, readable	Flexible, nested	Fast analytics, compact, typed

Item	CSV	JSON/JSONL	Parquet
Cons	Weak schema	Verbose, costly parsing	Not human-readable, poor row updates
Best use	Small exchange	APIs/events	Lakes, features, training tables

SQL

1. Overview

SQL is a declarative language for defining, querying, transforming, and controlling relational data. In ML systems it creates training cohorts, labels, point-in-time features, monitoring tables, and experiment analyses. “Declarative” means specifying the desired result while the database optimizer chooses scans, joins, and execution order.

2. Intuition

SQL is like describing the final set of index cards you want—“active customers, grouped by country, with average spend”—rather than prescribing every loop. The optimizer chooses a physical route using table statistics and indexes.

3. Prerequisites

- Sets, relations, predicates, null/three-valued logic, keys, and basic database concepts.
- **SELECT**, **FROM**, **WHERE**, **GROUP BY**, and ordering.
- Awareness of ML leakage and observation/prediction timestamps.

4. Core Concepts

Subtopic	Meaning / why it matters	Example	Interview angle
Projection/filter	Choose columns and rows.	SELECT age FROM users WHERE active	WHERE vs HAVING
Keys/constraints	Encode identity and valid relationships.	PRIMARY KEY (user_id)	Surrogate vs natural key
NULL logic	Unknown is neither equal nor unequal.	x IS NULL	Why does x = NULL fail?
CTE/subquery	Name intermediate relations for clarity.	WITH cohort AS (...)	CTE materialization varies by engine
Window function	Computes across related rows without collapsing them.	ROW_NUMBER() OVER (...)	Top-N per group / latest record
Transactions	Atomic, consistent, isolated, durable changes.	Publish a feature-table version.	Explain ACID/isolation

Subtopic	Meaning / why it matters	Example	Interview angle
Index	Auxiliary structure trading write/storage cost for lookup speed.	B-tree on (<code>user_id</code> , <code>event_time</code>)	Composite-index order
Query plan	Physical operators selected by optimizer.	scan → hash join → aggregate	How to diagnose a slow query

5. Algorithm / Working Process

1. Parse SQL and resolve table/column names.
2. Produce a logical relational plan: selections, projections, joins, and aggregates.
3. Rewrite it using equivalences such as filter pushdown.
4. Estimate cardinalities and costs using statistics.
5. Select physical scans, join algorithms, ordering, and parallelism.
6. Execute operators, possibly spilling or shuffling, then return/write the result.

For ML extraction: define an entity population and cutoff time, generate labels after the cutoff, compute features only from information available at or before it, validate uniqueness, and snapshot the output.

6. Mathematical Foundation

Relational algebra maps common clauses to operations:

$$\text{SELECT } a \text{ FROM } R \text{ WHERE } p \quad \leftrightarrow \quad \pi_a(\sigma_p(R)).$$

SQL uses three-valued logic: a predicate involving `NULL` may be `UNKNOWN`, and `WHERE` retains only `TRUE`. Selectivity is

$$s(P) = \frac{|\sigma_P(R)|}{|R|},$$

which helps estimate plan cost. For a binary classifier, a point-in-time label might be

$$y_i = \mathbf{1}\{\exists e : t_i < t_e \leq t_i + H\},$$

while every feature must use events with $t_e \leq t_i$.

7. Practical Implementation

```
-- One point-in-time training row per user and cutoff.
WITH feature_base AS (
  SELECT
    c.user_id,
    c.cutoff_time,
    COUNT(e.event_id) AS events_30d,
    COALESCE(SUM(e.amount), 0) AS spend_30d
  FROM cutoffs AS c
  LEFT JOIN events AS e
    ON e.user_id = c.user_id
```

```

        AND e.event_time > c.cutoff_time - INTERVAL '30 days'
        AND e.event_time <= c.cutoff_time
    GROUP BY c.user_id, c.cutoff_time
),
labels AS (
    SELECT
        c.user_id,
        c.cutoff_time,
        CASE WHEN COUNT(ch.churn_time) > 0 THEN 1 ELSE 0 END AS label
    FROM cutoffs AS c
    LEFT JOIN churn_events AS ch
        ON ch.user_id = c.user_id
        AND ch.churn_time > c.cutoff_time
        AND ch.churn_time <= c.cutoff_time + INTERVAL '30 days'
    GROUP BY c.user_id, c.cutoff_time
)
SELECT f.*, l.label
FROM feature_base AS f
JOIN labels AS l
    ON l.user_id = f.user_id AND l.cutoff_time = f.cutoff_time;

```

8. Code Explanation

The cohort table defines entity and cutoff. The feature join has both lower and upper time bounds and excludes future events. Labels intentionally use the following 30 days. **LEFT JOIN** retains users with no activity; **COALESCE** turns an empty monetary aggregate into zero. Joining on both keys prevents mixing multiple cutoffs for one user.

9. Training / Evaluation

SQL itself is not trained. Validate extracted data using row-count reconciliation, key uniqueness, null/range checks, class balance, temporal boundaries, and distribution comparisons against serving features. Use **EXPLAIN (ANALYZE, BUFFERS)** or the engine equivalent to compare estimated and actual rows, elapsed time, spills, and scanned bytes. Evaluate model data using time-based splits when future deployment is simulated.

10. Complexity and Cost

- Full scan: $O(N)$; indexed point lookup commonly $O(\log N)$ plus matched rows.
- Sort: $O(N \log N)$; hash aggregation/join expected $O(N)$ with sufficient memory.
- Network shuffle is often more expensive than CPU in distributed SQL.
- **SELECT ***, unbounded windows, repeated scans, and non-sargable predicates increase cost.
- Query plans depend on statistics and physical layout, so asymptotics alone are insufficient.

11. Common Use Cases

Cohort creation, feature computation, label construction, deduplication, missing-value summaries, drift reports, A/B-test analysis, model monitoring, and warehouse transformations.

12. Common Mistakes

- Leakage through future rows or mutable “current status” columns.
- **NOT IN** with a subquery containing **NULL**; prefer **NOT EXISTS** when semantics fit.
- Filtering the right table in **WHERE** after a **LEFT JOIN**, unintentionally making it inner.
- Assuming output order without **ORDER BY**.
- Joining at mismatched grains and duplicating training rows.
- Using **SELECT DISTINCT** to hide a bad join.
- Applying functions to indexed columns so predicates cannot use indexes/pruning.

13. Edge Cases / Limitations

SQL dialects differ in intervals, date arithmetic, arrays, merge syntax, and null ordering. Transactions and constraints are weaker or different in some analytical engines. Floating-point sums may vary with execution order. Slowly changing dimensions, late-arriving events, time zones, and mutable sources complicate reproducible features.

14. Variations

- OLTP SQL: normalized, indexed, low-latency reads/writes; foundational placement knowledge.
- OLAP SQL: columnar, distributed scans and aggregates; central to analytics/ML.
- ANSI SQL plus dialects: PostgreSQL, BigQuery, Snowflake, Spark SQL.
- dbt-style SQL transformation: versioned models, tests, lineage; valuable for data/ML engineering projects.

15. Related Topics

Joins combine entities; aggregations summarize behavior; ETL schedules SQL transformations; warehouses execute analytical SQL; feature pipelines add point-in-time correctness and online/offline consistency.

16. Interview Questions

1. **WHERE vs HAVING?** **WHERE** filters input rows before grouping; **HAVING** filters groups after aggregation.
2. **Window function vs GROUP BY?** Windows retain row grain; grouping collapses rows to one per group.
3. **Why does `col = NULL` not work?** It evaluates to **UNKNOWN**; use **IS NULL**.
4. **What is a CTE?** A named query expression that improves composition; whether it is materialized depends on the engine.
5. **How do you get the latest row per user?** Rank with **ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_time DESC)** and keep rank one.
6. **What makes a query sargable?** Its predicate can use an index or pruning structure, such as a range on the raw indexed column.
7. **How do you debug slow SQL?** Inspect the actual plan, cardinality errors, scans, join method, spills, skew, pruning, and repeated work.
8. **What is leakage in SQL features?** A feature uses information unavailable at prediction time.
9. **UNION vs UNION ALL?** **UNION** removes duplicates, typically with sort/hash work; **UNION ALL** concatenates.
10. **Why are constraints useful for ML?** They prevent invalid identities and relationships before silent corruption reaches training.

17. Practice Tasks

- Code: write top-three purchases per user using a window function.
- Dataset project: construct a churn cohort with point-in-time features.
- Experiment: compare plans before/after a composite index or partition filter.
- Debugging: locate row multiplication in a multi-table feature query.
- Extension: write assertions for uniqueness, temporal bounds, and class balance.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Churn feature mart	Builds cutoff-safe training rows	PostgreSQL/dbt; retail events	Shows leakage-aware SQL
Query plan clinic	Benchmarks indexes and query rewrites	PostgreSQL; generated data	Demonstrates performance reasoning
Experiment analyzer	Computes uplift and confidence summaries	DuckDB/SQL; A/B data	Connects statistics and SQL

19. Quick Revision

- Key idea: declare relations; optimizer selects physical execution.
- Main expression: $\pi(\sigma(R))$ for projection after selection.
- Metrics: rows/scanned bytes, latency, spill, estimate error.
- Trap: wrong grain and temporal leakage.
- Interview one-liner: "Correct ML SQL starts by fixing entity, grain, and cutoff time before computing features."

20. Final Cheat Sheet

Item	Summary
Input/output	Relations in, relation out
Main steps	Parse → logical optimize → physical plan → execute
Key tuning	indexes, partitions, stats, join order, projections
Metrics	latency, scanned bytes, rows, shuffle/spill
Pros/cons	expressive and optimizable / dialect and plan complexity
Best uses	cohorts, labels, features, analysis, monitoring

Joins

1. Overview

A join combines records from two relations using a condition. Joins connect labels, entities, events, embeddings, and predictions, but are also a major source of duplicate rows, leakage, skew, and distributed shuffle cost. Correctness requires knowing each table's grain and key cardinality before joining.

2. Intuition

Imagine matching two decks of cards by customer ID. If each deck has one card per customer, matching is one-to-one. If the purchase deck has ten cards per customer, joining it to five support tickets creates fifty combinations unless one side is aggregated first.

3. Prerequisites

Primary/foreign keys, set and bag semantics, NULL behavior, table grain, Big-O notation, hashing, sorting, and data partitioning.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Inner join	Keeps matching pairs only.	Users with orders.	Missing-key effect on cohort
Left join	Keeps every left row, fills unmatched right fields with NULL.	All users plus optional profile.	Filter placement
Full join	Keeps matched and unmatched rows from both sides.	Source reconciliation.	Detect missing records
Semi/anti join	Keeps left rows with/without a match, not right columns.	Eligible/non-eligible users.	EXISTS / NOT EXISTS
Cross join	Cartesian product.	Hyperparameter grid.	Explosion risk
Join cardinality	One-to-one, one-to-many, or many-to-many.	Customer to events.	Predict output row count
Equi/range/as-of	Equality, inequality/range, or nearest earlier-time match.	Feature latest before prediction.	Point-in-time correctness
Physical join	Nested-loop, hash, sort-merge, broadcast.	Broadcast small dimension.	Choose based on size/order

5. Algorithm / Working Process

Hash join: build a hash table on the smaller input keyed by join key, scan the other input, probe matching buckets, and emit pairs. **Sort-merge join:** sort/repartition both sides by key, advance through ordered runs, and emit matching groups. **Broadcast join:** copy a small table to each worker and join locally, avoiding a large shuffle. For correctness, first document grain and expected relationship, check duplicate/null keys, join, then assert output cardinality and unmatched rates.

6. Mathematical Foundation

For relations R and S joined on predicate θ :

$$R \bowtie_{\theta} S = \{(r, s) : r \in R, s \in S, \theta(r, s)\}.$$

For key k , let frequencies be $f_R(k)$ and $f_S(k)$. Inner-join output size is exactly

$$|R \bowtie S| = \sum_k f_R(k) f_S(k).$$

This formula exposes many-to-many explosions and hot-key skew. Hash join expected time is $O(|R| + |S| + |output|)$ and memory is $O(\min(|R|, |S|))$ for the build side. Sort-merge is $O(R \log R + S \log S)$ if sorting is required.

7. Practical Implementation

```
import pandas as pd

users = pd.DataFrame({"user_id": [1, 2, 3], "country": ["IN", "US", "IN"]})
scores = pd.DataFrame({"user_id": [1, 2], "score": [0.9, 0.3]})

# validate prevents silent many-to-many row multiplication.
joined = users.merge(scores, on="user_id", how="left", validate="one_to_one",
indicator=True)
assert len(joined) == len(users)
assert joined["user_id"].is_unique
print(joined["_merge"].value_counts())

events = pd.DataFrame({
    "user_id": [1, 1, 2],
    "event_time": pd.to_datetime(["2026-01-01", "2026-01-05", "2026-01-02"]),
    "value": [10, 20, 30],
}).sort_values("event_time")
predictions = pd.DataFrame({
    "user_id": [1, 2],
    "prediction_time": pd.to_datetime(["2026-01-04", "2026-01-03"]),
}).sort_values("prediction_time")

point_in_time = pd.merge_asof(
    predictions, events,
    left_on="prediction_time", right_on="event_time",
    by="user_id", direction="backward",
)
assert (point_in_time["event_time"] <= point_in_time["prediction_time"]).all()
```

8. Code Explanation

The first merge declares the expected one-to-one contract and records which side supplied each row. The as-of join selects the latest event no later than prediction time for the same user. Sorting is required. The temporal assertion is a compact guard against future-data leakage.

9. Training / Evaluation

There is no join training. Evaluate join correctness with left/right uniqueness, expected output bounds, unmatched rate, multiplicity distribution, per-key frequency, null-key count, and temporal validity. Downstream model metrics cannot prove join correctness: leakage may improve validation results. Compare online and offline joined features on sampled entity/timestamp pairs.

10. Complexity and Cost

- Nested-loop: $O(RS)$ generally; efficient with a tiny outer side and indexed inner lookups.
- Hash: expected $O(R + S + output)$; requires build-side memory and handles equality only.
- Sort-merge: sorting cost plus linear merge; good for large ordered/range-compatible data.
- Distributed joins may shuffle both tables, with network/disk cost proportional to input size.
- Skew causes one partition to dominate completion time; salting or special skew handling may help.

11. Common Use Cases

Attach labels to features, enrich events with dimensions, retrieve latest feature state, reconcile sources, filter eligible entities with semi joins, find missing predictions with anti joins, and associate recommendations with outcomes.

12. Common Mistakes

- Joining two event-grain tables directly and causing a many-to-many explosion.
- Assuming declared uniqueness without testing actual data.
- Joining on names or rounded timestamps instead of stable keys.
- Allowing post-cutoff events in training features.
- Turning a left join into an inner join with `WHERE right.col = ....`
- Treating nulls as ordinary equal keys when engine semantics differ.
- Broadcasting a table that is too large for executor memory.

13. Edge Cases / Limitations

Duplicate keys, null keys, key type differences, Unicode normalization, floating-point keys, late events, and slowly changing dimensions complicate matching. As-of joins need explicit direction, tolerance, tie-breaking, and time-zone rules. Highly skewed keys can defeat otherwise linear distributed algorithms.

14. Variations

- Interval join: match when a timestamp lies inside `[valid_from, valid_to)`; important for slowly changing dimensions.
- Fuzzy/probabilistic linkage: match imperfect names/addresses; project/research relevant and requires confidence evaluation.
- Spatial join: match points and regions; domain-specific.
- Bloom-filter join reduction: probabilistically remove impossible keys before shuffle; advanced systems interview topic.

15. Related Topics

Aggregations repair grain before a join. SQL expresses logical joins; Spark selects and executes distributed physical joins. Feature pipelines use as-of joins to prevent leakage. Warehouses use statistics and clustering to

optimize join plans.

16. Interview Questions

1. **Inner vs left join?** Inner retains matches; left retains every left row plus matches or nulls.
2. **What causes row explosion?** Repeated keys on both sides; each pair of matching occurrences is emitted.
3. **How do you detect it?** Inspect key counts, declare cardinality, and compare expected versus actual output rows.
4. **Hash vs sort-merge join?** Hash is fast for equality with memory; sort-merge suits large ordered data and some range patterns.
5. **What is a broadcast join?** Replicate a small side to workers so the large side need not shuffle.
6. **Semi join?** Return left rows that have a right match without returning right columns or multiplying matches.
7. **Why use `NOT EXISTS` for anti joins?** It has clear null semantics compared with `NOT IN` on nullable data.
8. **What is an as-of join?** Match each record to the nearest qualifying record, usually the latest one not after its timestamp.
9. **How does skew affect a join?** One hot key sends excessive work/data to one partition, creating a straggler or OOM.
10. **How do joins create ML leakage?** A label or feature row from after prediction time is matched into training input.

17. Practice Tasks

- Code: implement and validate a one-to-many customer/order join.
- Dataset project: point-in-time join transactions to monthly prediction cutoffs.
- Experiment: compare hash/broadcast/shuffle plans in Spark.
- Debugging: explain why 10K and 20K row inputs produced 2M rows.
- Extension: handle a slowly changing customer tier with an interval join.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Join auditor	Profiles cardinality, unmatched rows, and hot keys	Python/DuckDB	Prevents silent dataset corruption
Point-in-time feature builder	Performs cutoff-safe as-of joins	pandas/Spark; event data	Strong ML-platform signal
Entity resolver	Links noisy customer records with evaluated confidence	Python, record linkage	Shows practical data science

19. Quick Revision

- Key idea: join correctness begins with grain and cardinality.
- Formula: $\text{output} = \sum_k f_R(k) f_S(k)$.
- Metrics: unmatched rate, duplication factor, hot-key share, shuffle bytes.

- Trap: many-to-many multiplication and temporal leakage.
- Interview one-liner: "Before joining, state the grain of both tables and the expected key relationship."

20. Final Cheat Sheet

Item	Summary
Input/output	Two relations → combined or filtered relation
Main steps	validate keys → choose logical join → execute physical join → assert grain
Key tuning	build side, broadcast threshold, partitioning, skew handling
Metrics	rows, match rate, multiplicity, shuffle/spill
Pros/cons	expressive integration / leakage, explosion, network cost
Best uses	entity enrichment, labels, point-in-time features, reconciliation

Aggregations

1. Overview

Aggregation reduces multiple rows into summaries such as count, sum, mean, quantile, or distinct count, usually per group and time window. It converts raw behavior into ML features, monitoring statistics, and evaluation metrics. The essential design choice is the output grain: one row per user, user-day, session, or another key.

2. Intuition

A transaction log is a long receipt. Aggregation turns it into a customer summary: purchases in 7 days, average basket value, days since last order, or number of distinct categories.

3. Prerequisites

Descriptive statistics, SQL grouping, null handling, keys and grain, time windows, numerical stability, and basic distributed partitioning.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Group aggregate	One result per group.	Spend per user.	Input/output grain
Window aggregate	Summary attached to each row.	Rolling 7-day count.	Window vs group
Additive statistic	Partial results combine exactly.	count, sum, min, max	Map-side combine

Subtopic	Meaning / why	Example	Interview angle
Algebraic statistic	Fixed-size partial state combines.	mean via (sum, count)	Distributed average
Holistic statistic	Exact state may grow with data.	median, exact distinct	Approximation tradeoff
Event-time window	Group by when event occurred.	Requests per event minute.	Late data/watermark
Conditional aggregate	Summarize matching rows.	Failed-payment count.	SQL CASE/FILTER
Approximate sketch	Compact probabilistic summary.	HyperLogLog distinct count.	Error-memory tradeoff

5. Algorithm / Working Process

1. Define entity key, cutoff, window, and null/empty-group semantics.
2. Filter events to the allowed time interval.
3. Map each event to a grouping key and partial state.
4. Combine partial states locally when possible.
5. Shuffle by group key in distributed systems.
6. Merge states and finalize metrics.
7. Validate output uniqueness, ranges, and temporal boundaries.

For a mean, workers emit (sum, count), reducers add both, then divide; averaging worker averages is wrong when partition sizes differ.

6. Mathematical Foundation

For group G_k with n_k values:

$$\bar{x}_k = \frac{1}{n_k} \sum_{i \in G_k} x_i, \quad s_k^2 = \frac{1}{n_k - 1} \sum_{i \in G_k} (x_i - \bar{x}_k)^2.$$

Variance can be merged stably using count, mean, and squared-deviation state rather than $E[X^2] - E[X]^2$, which may suffer cancellation. An exponentially weighted statistic is

$$z_t = \alpha x_t + (1 - \alpha) z_{t-1}, \quad 0 < \alpha \leq 1.$$

For an approximate distinct estimator, report relative error and confidence rather than treating the estimate as exact.

7. Practical Implementation

```
import pandas as pd

events = pd.DataFrame({
    "user_id": [1, 1, 1, 2],
```

```

    "event_time": pd.to_datetime([
        "2026-01-01", "2026-01-03", "2026-01-10", "2026-01-02"
    ]),
    "amount": [10.0, 30.0, 20.0, 5.0],
    "category": ["A", "B", "A", "A"],
})
cutoff = pd.Timestamp("2026-01-11")

eligible = events[
    (events["event_time"] > cutoff - pd.Timedelta(days=7))
    & (events["event_time"] <= cutoff)
]
features = (
    eligible.groupby("user_id", as_index=False)
    .agg(
        txn_count_7d=("amount", "size"),
        spend_7d=("amount", "sum"),
        avg_amount_7d=("amount", "mean"),
        categories_7d=("category", "nunique"),
        last_event_time=("event_time", "max"),
    )
)
features["recency_days"] = (cutoff - features["last_event_time"]).dt.days
assert features["user_id"].is_unique
assert (features["last_event_time"] <= cutoff).all()

```

8. Code Explanation

The explicit half-open-like time policy excludes events older than seven days and includes those at cutoff. Named aggregation makes feature meanings visible. `size` counts rows even if `amount` is null; `count` would count non-null values. The assertions protect the promised grain and cutoff.

9. Training / Evaluation

Aggregations are deterministic transforms. Evaluate their feature value through coverage, null/zero rate, distribution, freshness, stability, and downstream validation lift. Fit any learned boundaries—quantile bins, target encodings, normalization parameters—on training data only. For streaming/offline parity, compare the same entity/window at the same cutoff.

10. Complexity and Cost

- Hash group-by: expected $O(N)$ time and $O(K)$ state, potentially spilling when K is large.
- Sort aggregation: $O(N \log N)$ if unsorted, then linear reduction.
- Rolling exact windows require state proportional to active window events unless a mergeable bucketed summary is used.
- Exact `COUNT(DISTINCT)` may need large sets/shuffles; sketches trade bounded error for much lower memory.
- GPUs are rarely required; CPU, memory, shuffle, and state-store I/O dominate.

11. Common Use Cases

RFM/customer features, click counts, mean sensor values, class and missingness reports, monitoring latency percentiles, session summaries, fraud velocity features, and recommendation interaction statistics.

12. Common Mistakes

- Omitting the grouping key from the intended grain.
- Averaging averages without weighting by counts.
- Confusing `COUNT(*)`, `COUNT(column)`, and distinct count.
- Treating "no events" as null in one pipeline and zero in another.
- Leakage from windows extending beyond cutoff.
- Using exact median/distinct count at unnecessary scale.
- Ignoring extreme values that dominate sums and means.

13. Edge Cases / Limitations

Empty groups, all-null groups, integer overflow, floating-point non-associativity, duplicate events, late data, daylight-saving boundaries, and hot groups need explicit handling. Aggregation discards information; identical summaries can hide very different sequences, so sequence models may require raw or ordered features.

14. Variations

- Tumbling, hopping, and session windows: fixed non-overlapping, overlapping, and gap-defined windows; important for streaming interviews.
- Exponential decay: prioritizes recent behavior without retaining a hard window.
- Robust aggregates: median, trimmed mean, winsorized mean; useful with outliers.
- Sketches: HyperLogLog, t-digest, Count-Min Sketch; advanced but valuable for scale.

15. Related Topics

Aggregations define feature-pipeline state, execute through SQL/Spark group-by, and require Kafka/Airflow semantics for complete windows. Streaming uses watermarks and state stores; distributed internals explain combiners, shuffles, and skew.

16. Interview Questions

1. **`COUNT(*)` vs `COUNT(col)`?** The former counts rows; the latter counts non-null values.
2. **GROUP BY vs window function?** Grouping changes grain; a window retains rows.
3. **How do you compute a distributed mean?** Merge sums and counts, then divide.
4. **Why is median hard to distribute exactly?** Exact computation may require retaining or ordering all values.
5. **What is a tumbling window?** Fixed, non-overlapping time buckets.
6. **What is a watermark?** A declaration of progress in event time used to bound late-data state.
7. **How do you handle users with no events?** Start from the entity cohort, left join aggregates, and apply documented defaults.
8. **Why can sums differ across runs?** Floating-point addition is not associative and distributed reduction order may change.

9. **How do you reduce group-by shuffle?** Project/filter early and use mergeable local partial aggregation.
10. **Feature count spikes unexpectedly—what do you check?** Duplicates, replay, changed window boundaries, time zones, upstream join multiplication, and late events.

17. Practice Tasks

- Code: build 1/7/30-day recency-frequency-monetary features.
- Dataset project: aggregate taxi trips by zone/hour and predict demand.
- Experiment: compare exact and approximate distinct counts.
- Debugging: fix a distributed “average of averages” bug.
- Extension: implement event-time windows with late-event updates.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Fraud velocity features	Counts/cardinality over short windows	Spark/Kafka; synthetic payments	Real-time ML relevance
Demand feature mart	Hourly zone statistics	SQL/dbt; taxi trips	Grain and temporal reasoning
Sketch benchmark	Tests approximate cardinality/quantiles	Python/Spark	Demonstrates scale tradeoffs

19. Quick Revision

- Key idea: reduce rows to a declared grain using mergeable state where possible.
- Formula: mean is `sum/count`, not mean of partition means.
- Metrics: coverage, null rate, state size, shuffle, approximation error.
- Trap: wrong window/cutoff and missing empty groups.
- Interview one-liner: “A production aggregate is defined by key, window, cutoff, null semantics, and update policy.”

20. Final Cheat Sheet

Item	Summary
Input/output	Event rows → one summary per group/window
Main steps	filter → key → partial aggregate → shuffle → merge/finalize
Key tuning	keys, window size, state, partitions, approximation error
Metrics	groups, state bytes, shuffle, lateness, coverage
Pros/cons	compact, useful features / information loss, skew/state cost
Best uses	behavioral features, metrics, monitoring, reporting

ETL

1. Overview

ETL—Extract, Transform, Load—moves data from sources into a trusted destination. In ML it ingests operational data, validates and standardizes it, computes reusable features/labels, and publishes versioned training or monitoring tables. ELT loads raw data first and transforms inside a scalable warehouse/lakehouse. The distinction matters less than correctness, lineage, repeatability, and ownership.

2. Intuition

ETL resembles a food-processing line: collect ingredients, inspect and clean them, apply a recipe, then package a traceable batch. If the same raw ingredients and recipe cannot reproduce yesterday's output, the training dataset is not auditable.

3. Prerequisites

File/database access, schemas, SQL/Python transforms, timestamps, idempotency, partitions, basic orchestration, tests, and ML train/serve leakage concepts.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Extract	Read source while limiting load and preserving change order.	Incremental orders by update time.	Full vs incremental extract
Transform	Clean, normalize, validate, enrich, and aggregate.	Convert cents to rupees.	Where should business logic live?
Load	Publish to destination safely.	Atomic replace of date partition.	Append, merge, overwrite
ELT	Load raw first, transform later in compute engine.	Warehouse staging tables.	ETL vs ELT tradeoff
Idempotency	Repeating a run yields the same final state.	Upsert by event ID.	Retry without duplicates
CDC	Capture inserts, updates, and deletes from source logs.	Database WAL stream.	Ordering and tombstones
Backfill	Recompute historical intervals with controlled scope.	Repair last month's labels.	Avoid harming current SLA
Lineage	Track source, code/version, run, and outputs.	Model dataset tied to commit.	Reproducibility

5. Algorithm / Working Process

1. Define the source contract, destination grain/schema, watermark, and SLA.

2. Extract a bounded snapshot or ordered changes.
3. Land immutable raw data with ingestion metadata.
4. Validate schema and quarantine malformed records.
5. Apply deterministic transformations and deduplicate.
6. Run row-level and aggregate quality checks.
7. Write to staging, then atomically publish/merge.
8. Record lineage, metrics, watermark, and status; retry safely or backfill when needed.

6. Mathematical Foundation

For deterministic transform T and load operation L , a rerunnable pipeline aims for state idempotence:

$$L(T(X), L(T(X), S)) = L(T(X), S).$$

Incremental processing over watermark interval $(w_{old}, w_{new}]$ reads

$$\Delta X = \{x : w_{old} < t(x) \leq w_{new}\}.$$

A late-arrival allowance δ may reread from $w_{old} - \delta$ and deduplicate by stable key. Reconciliation often checks $N_{in} = N_{accepted} + N_{rejected}$ and expected sums/counts across stages.

7. Practical Implementation

```
from pathlib import Path
import pandas as pd

def run_etl(source_csv: str, output_dir: str, run_date: str) -> None:
    raw = pd.read_csv(source_csv, dtype={"order_id": "string", "user_id":
"string"})
    required = {"order_id", "user_id", "amount_cents", "event_time"}
    if missing := required - set(raw.columns):
        raise ValueError(f"missing columns: {sorted(missing)}")

    data = raw.copy()
    data["event_time"] = pd.to_datetime(data["event_time"], utc=True,
errors="coerce")
    data["amount"] = pd.to_numeric(data["amount_cents"], errors="coerce") / 100
    data = data.drop_duplicates("order_id", keep="last")

    valid = data[
        data["order_id"].notna()
        & data["user_id"].notna()
        & data["event_time"].notna()
        & data["amount"].ge(0)
    ][["order_id", "user_id", "event_time", "amount"]]
    if valid["order_id"].duplicated().any():
        raise ValueError("order_id is not unique")

    # Deterministic partition replacement makes a retry idempotent.
    target = Path(output_dir) / f"run_date={run_date}" / "orders.parquet"
    target.parent.mkdir(parents=True, exist_ok=True)
```

```
valid.sort_values("order_id").to_parquet(target, index=False)

run_etl("orders.csv", "curated_orders", "2026-08-12")
```

8. Code Explanation

The function explicitly fixes identifier types, validates required columns, normalizes timestamps to UTC, converts units, and deduplicates on the business key. It fails on a broken uniqueness invariant. Writing one deterministic partition path means a retry replaces the same logical batch instead of appending duplicates. Production object stores usually publish through a transactional table format or staged commit rather than relying on file replacement.

9. Training / Evaluation

ETL is evaluated with freshness, completeness, validity, uniqueness, consistency, reconciliation, run duration, retry rate, and recovery objectives. Dataset quality also needs label prevalence, feature distributions, point-in-time correctness, and split integrity. Test transformations with fixed input/output cases, integration tests against representative schemas, and production canaries; a successful job is not proof of correct data.

10. Complexity and Cost

Most row transforms are $O(N)$; deduplication is expected $O(N)$ with hashing or $O(N \log N)$ with sorting. Full reload cost grows with history; incremental cost grows with changes plus overlap. I/O, warehouse scan bytes, shuffle, and serialization usually dominate. Backfills compete with scheduled runs, so isolate pools/queues and bound partitions.

11. Common Use Cases

Operational database replication, raw-event cleanup, privacy redaction, feature-table materialization, image/NLP metadata manifests, label creation, model-monitoring tables, and migration between schemas.

12. Common Mistakes

- Incrementing by processing time and permanently missing late source updates.
- Retrying an append-only load without deduplication.
- Mutating raw data instead of keeping a replayable landing layer.
- Silent coercion that turns parse failures into nulls without quarantine metrics.
- In-place partial writes exposed to readers.
- No delete/tombstone handling in CDC.
- Recomputing preprocessing using validation/test information.

13. Edge Cases / Limitations

Source schema drift, out-of-order CDC, source deletions, late arrivals, daylight-saving boundaries, partial upstream snapshots, and poison records require policies. Exactly-once effects across unrelated external systems are difficult; idempotent sinks plus at-least-once processing are more common. Historical backfills may reproduce current logic, not logic as it existed then, unless code and dependencies are versioned.

14. Variations

- Full refresh: simplest and safest for small tables; costly at scale.
- Incremental append: efficient for immutable facts; needs watermark handling.
- Upsert/merge: handles corrections; requires stable keys and sink support.
- CDC: low-latency source changes including deletes; operationally advanced.
- Reverse ETL: sends warehouse-derived data back to operational tools; less central to ML placements.

15. Related Topics

Airflow orchestrates ETL dependencies; Spark executes large transformations; warehouses/lakehouses store outputs; data-quality checks guard transitions; feature pipelines add point-in-time and train/serve consistency requirements.

16. Interview Questions

1. **ETL vs ELT?** ETL transforms before loading the target; ELT lands data then transforms using target compute.
2. **What makes an ETL job idempotent?** Reprocessing the same logical input leaves one identical result, commonly via partition replacement or keyed merge.
3. **Full vs incremental load?** Full reloads all data; incremental processes changes and needs watermarks, late-data, update, and delete semantics.
4. **What is CDC?** Capture of database changes from logs or triggers as ordered insert/update/delete events.
5. **How do you publish safely?** Write and validate staging output, then atomically commit/swap a partition or table snapshot.
6. **How do you handle bad rows?** Quarantine with reason and source metadata, measure the rate, alert by threshold, and preserve replayability.
7. **What is a watermark in batch ETL?** The durable high point through which source changes have been successfully processed.
8. **How do you backfill?** Parameterize partitions, pin code/config, isolate resources, run quality checks, and publish in controlled increments.
9. **Why preserve raw data?** It enables replay, debugging, audit, and new transforms without re-querying fragile sources.
10. **How does ETL cause leakage?** Features or preprocessing use records/statistics that were unavailable at the historical prediction cutoff.

17. Practice Tasks

- Code: build an idempotent CSV-to-Parquet daily load.
- Dataset project: ingest public transactions into raw, clean, and feature layers.
- Experiment: compare full refresh with incremental overlap/dedup.
- Debugging: repair duplicate rows caused by a retry after partial failure.
- Extension: process CDC updates and tombstones into a current-state table.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Retail ETL	Raw-to-curated orders with rejects and reconciliation	Python, DuckDB, Parquet	End-to-end reliability
CDC replica demo	Applies ordered changes and deletes	PostgreSQL/Debezium or synthetic log	Advanced ingestion semantics
ML dataset builder	Produces versioned cutoff-safe features and labels	SQL, dbt/Airflow	Direct ML engineering relevance

19. Quick Revision

- Key idea: reproducibly move source data into a validated, published contract.
- Formula: incremental interval (w_{old}, w_{new}] plus overlap for late data.
- Metrics: freshness, completeness, validity, duration, rejects.
- Trap: non-idempotent retry and silent schema drift.
- Interview one-liner: "A robust ETL job is deterministic, idempotent, observable, and replayable."

20. Final Cheat Sheet

Item	Summary
Input/output	Source snapshot/changes → curated versioned table
Main steps	extract → land → validate → transform → test → publish
Key tuning	watermark, overlap, batch size, partitions, retries
Metrics	freshness, counts, rejects, runtime, reconciliation
Pros/cons	reusable trusted data / latency and operational complexity
Best uses	ingestion, cleaning, feature/label materialization

Data Pipelines

1. Overview

A data pipeline is the complete graph of producers, transformations, stores, quality gates, and consumers that moves data through a system. ETL is one pipeline pattern; a pipeline additionally covers dependencies, scheduling/event triggers, versioning, observability, failure recovery, and service-level objectives. ML pipelines feed training, feature serving, batch inference, evaluation, and monitoring.

2. Intuition

Think of a railway network, not a single train. Tracks represent dependencies, stations are datasets or services, timetables are schedules, signals are quality gates, and rerouting/recovery determine whether one failure disrupts everything.

3. Prerequisites

DAGs, ETL, storage, batch/stream concepts, idempotency, APIs, queues, version control, testing, and basic reliability terminology.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
DAG	Directed acyclic dependency graph.	ingest → clean → features → train	Why no cycles?
Data contract	Producer/consumer agreement on schema and semantics.	<code>user_id</code> non-null string.	Schema compatibility
Idempotent task	Safe retry for same logical interval.	Replace date partition.	Failure recovery
Checkpoint/watermark	Durable progress marker.	Last processed offset/date.	Resume semantics
SLO/SLA	Objective/commitment for freshness, quality, availability.	99% features by 07:00.	Measuring freshness
Backpressure	Slow consumers limit or accumulate upstream work.	Kafka lag rises.	Capacity handling
Lineage	Trace outputs to inputs/code/runs.	Prediction to feature snapshot.	Incident blast radius
Observability	Metrics, logs, traces, data profiles.	Row count and lag dashboard.	Job success vs data health

5. Algorithm / Working Process

1. Define data products, owners, consumers, grain, contract, latency, and SLO.
2. Draw the smallest dependency DAG and choose batch, streaming, or hybrid execution.
3. Make every boundary replayable and every sink idempotent.
4. Add validation before publication, not merely after consumption.
5. Persist run metadata and checkpoints.
6. Orchestrate dependencies with bounded retries and alerts.
7. Observe both infrastructure and data behavior.
8. Support controlled backfills, schema migrations, and rollback/snapshot selection.

6. Mathematical Foundation

For serial stages with latencies L_i , end-to-end latency is approximately

$$L_{e2e} = \sum_i L_i + L_{queue} + L_{schedule}.$$

For independent parallel branches, the critical-path latency is the maximum path sum, not total work. If stage availabilities are independent and all are required, approximate pipeline availability is

$$A_{\text{pipeline}} = \prod_i A_i,$$

showing why many fragile stages reduce reliability. Little's Law relates average in-flight work W , arrival rate λ , and time T : $W = \lambda T$.

7. Practical Implementation

```
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

@dataclass(frozen=True)
class Step:
    name: str
    run: Callable[[str], None]

def require_file(path: str) -> None:
    if not Path(path).exists():
        raise RuntimeError(f"required output missing: {path}")

def run_daily(date: str, steps: list[Step]) -> None:
    """A tiny local runner; production DAG tools add scheduling and durable state."""
    for step in steps:
        print(f"date={date} step={step.name} status=started")
        step.run(date)          # each step must be idempotent for this
    date
    print(f"date={date} step={step.name} status=succeeded")

# Example use:
# run_daily("2026-08-12", [Step("ingest", ingest), Step("validate", validate)])
```

8. Code Explanation

The runner deliberately contains only ordered execution and status logging; real scheduling belongs to Airflow or another orchestrator. Passing the logical date into every step makes partition scope explicit. The key contract is outside the runner: each step must produce a deterministic output for that date and fail if its quality gate is not satisfied.

9. Training / Evaluation

Evaluate pipelines through data SLO attainment, end-to-end freshness, throughput, success/retry rate, mean time to detect/recover, correctness after replay, lineage completeness, and cost per run/record. For ML, track training-serving skew, dataset version reproducibility, label delay, feature freshness, and prediction coverage. Chaos-test retry and partial-failure paths on non-production data.

10. Complexity and Cost

Total compute is the sum of task work, but wall time follows the critical path plus queues. Fine-grained tasks improve retry scope but add scheduler and storage overhead. Over-parallelization can saturate databases or APIs. Materializing every intermediate increases cost; recomputing everything increases latency. Persist boundaries that enable reuse, audit, or practical recovery.

11. Common Use Cases

Daily training datasets, media preprocessing, embedding generation, batch inference, feature materialization, model monitoring, event ingestion, document indexing for RAG, and experiment reporting.

12. Common Mistakes

- Treating task success as proof that output data is valid.
- One giant task with no observable/retry boundary, or hundreds of trivial tasks.
- Hidden dependencies on “latest” mutable tables.
- Retries that duplicate records or external effects.
- Coupling unrelated consumers to the same release schedule.
- No backfill isolation, ownership, or runbook.
- Training and serving implementations of the same feature diverge.

13. Edge Cases / Limitations

Partial upstream availability, delayed labels, out-of-order data, schema migration across mixed consumers, overlapping scheduled runs, quota limits, and poisoned partitions complicate pipeline logic. A DAG expresses dependencies but not semantic correctness. Cross-system atomicity is rare; design for reconciliation and replay.

14. Variations

- Scheduled batch DAG: simple, inspectable, placement-essential.
- Event-driven pipeline: lower latency but harder ordering/retry semantics.
- Streaming topology: continuous stateful computation with backpressure.
- Lambda architecture: batch and speed paths; powerful but duplicated logic.
- Kappa architecture: streaming log as primary path with replay; operationally simpler when viable.

15. Related Topics

ETL defines stage behavior; Airflow schedules batch DAGs; Kafka transports durable events; Spark processes batch or streams; data-quality systems gate publication; feature pipelines specialize the design for ML time semantics.

16. Interview Questions

1. **What is a data pipeline?** A managed graph that moves and transforms data with contracts, state, observability, and recovery.
2. **Why use a DAG?** It makes dependencies and runnable parallelism explicit while preventing circular scheduling.

3. **How do you make retries safe?** Scope work by logical interval and use idempotent writes/deduplication.
4. **What should be monitored?** Infrastructure plus freshness, volume, schema, distributions, rejects, and consumer-visible completeness.
5. **How do you choose task granularity?** Separate meaningful retry, ownership, quality, or reuse boundaries; avoid scheduler-noise tasks.
6. **What is backpressure?** Downstream capacity is below incoming rate, so lag/state grows or upstream must slow.
7. **How do you support reproducibility?** Pin source snapshots, code, configuration, dependencies, and output version metadata.
8. **Batch or streaming?** Choose the highest latency the use case tolerates; streaming adds state and operational cost.
9. **What is the critical path?** The longest dependent path determining earliest completion.
10. **How would you recover a corrupted week?** Find lineage/blast radius, fix/version logic, backfill isolated partitions, validate, and atomically republish.

17. Practice Tasks

- Code: implement a three-stage idempotent local DAG.
- Dataset project: raw events → validated features → model predictions.
- Experiment: measure critical path before/after parallelizing independent stages.
- Debugging: trace why a “green” pipeline published empty data.
- Extension: add run metadata, checkpoints, and partitioned backfill.

18. Project Ideas

Project	What it does	Stack/data	Resume value
RAG indexing pipeline	Parses, chunks, embeds, and versions documents	Airflow, Python, vector DB	Modern AI engineering
Batch scoring platform	Materializes features and publishes predictions	Spark/Parquet	Production ML lifecycle
Pipeline observability kit	Tracks freshness, counts, rejects, lineage	SQL, dashboard	Reliability and ownership

19. Quick Revision

- Key idea: a pipeline is computation plus dependencies, contracts, durable progress, and recovery.
- Formula: wall time follows the critical path.
- Metrics: freshness, throughput, SLO, lag, MTTR, cost.
- Trap: green jobs with wrong or stale data.
- Interview one-liner: “Design every pipeline stage around a contract, idempotent boundary, and observable output.”

20. Final Cheat Sheet

Item	Summary
Input/output	Source data/events → consumer-ready data products
Main steps	contract → DAG → transform → validate → publish → observe
Key tuning	granularity, parallelism, retries, checkpoints, retention
Metrics	critical-path latency, freshness, success, lag, cost
Pros/cons	automation/reproducibility / distributed failure complexity
Best uses	training, scoring, features, monitoring, indexing

Batch Processing

1. Overview

Batch processing operates on a finite, bounded collection of records, usually on a schedule or trigger. It powers nightly feature generation, historical training sets, embedding backfills, large inference jobs, and warehouse reports. Batch is often preferable when minute/hour/day latency is acceptable because bounded data and retries are easier to reason about than continuous streams.

2. Intuition

Batch processing is doing laundry after a basket fills; streaming is washing each item immediately. Waiting lets the machine process many items efficiently, but results arrive later.

3. Prerequisites

Partitions, files/tables, ETL, scheduling, parallelism, idempotency, distributed failure, and basic storage/compute cost concepts.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Bounded input	A finite snapshot or partition range.	Events for one date.	Batch vs stream
Partitioning	Split work by date/key/file for parallelism and retry.	<code>event_date=2026-08-12</code> .	Partition size choice
Incremental batch	Process only new/changed partitions.	Hourly micro-batch.	Watermarks/late data
Backfill	Re-run historical partitions.	Recompute embeddings.	Resource isolation
Atomic publication	Readers see old or new complete result.	Snapshot commit.	Partial-output safety

Subtopic	Meaning / why	Example	Interview angle
Straggler	A task much slower than peers.	Skewed customer partition.	Speculation/skew
Retry/checkpoint	Recompute failed scope rather than all work.	Retry one partition.	Idempotency
SLA	Completion deadline and quality target.	Ready before market opens.	Capacity planning

5. Algorithm / Working Process

1. Freeze the logical input boundary/snapshot.
2. Discover and prune required partitions/files.
3. Split input into tasks and schedule them across workers.
4. Read, transform, and optionally locally combine records.
5. Shuffle for joins/aggregations when required.
6. Write staged partition outputs.
7. Validate counts/schema/distributions.
8. Commit atomically and record processed partitions/checkpoints.

6. Mathematical Foundation

Ideal wall time with P balanced workers is

$$T_P \approx \frac{W}{P} + T_{overhead},$$

but the slowest task determines a stage: $T_{stage} = \max_j T_j$. Amdahl's law bounds speedup when fraction s is serial:

$$Speedup(P) = \frac{1}{s + (1 - s)/P}.$$

For arrival rate λ records/s and run interval B , a batch contains about λB records and compute must sustain average throughput above λ to avoid accumulating backlog.

7. Practical Implementation

```
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path
import pandas as pd

def score_partition(path: Path, model, output_root: Path) -> int:
    frame = pd.read_parquet(path)
    features = frame[["age", "spend_30d"]].fillna(0)
    frame["score"] = model.predict_proba(features)[ :, 1]
    target = output_root / path.name
    frame[["user_id", "score"]].to_parquet(target, index=False)
    return len(frame)
```

```
def batch_score(input_root: str, output_root: str, model) -> int:
    inputs = sorted(Path(input_root).glob("*.parquet"))
    out = Path(output_root)
    out.mkdir(parents=True, exist_ok=True)
    with ThreadPoolExecutor(max_workers=4) as pool:
        counts = list(pool.map(lambda p: score_partition(p, model, out), inputs))
    if sum(counts) == 0:
        raise ValueError("batch produced no predictions")
    return sum(counts)
```

8. Code Explanation

Each file is an independent retry unit, and the output filename deterministically corresponds to the input. Threads can overlap remote I/O; CPU-heavy Python transforms may require processes or a distributed engine. A production design writes to a run-specific staging snapshot, validates the full set, then publishes it atomically.

9. Training / Evaluation

Evaluate runtime, throughput, queue delay, completion SLA, processed/rejected counts, retry and straggler rates, output completeness, and cost. Batch inference adds prediction coverage, score distribution, model version, and comparison against a reference implementation. Historical training jobs must pin snapshots and use appropriate temporal splits.

10. Complexity and Cost

Row-local work is $O(N)$ and parallelizable. Global sorts are $O(N \log N)$ and joins/group-bys add shuffle. Memory depends on partition and operator working sets; spills increase disk I/O. CPUs are typical for ETL, while GPU batch inference/training benefits from sufficiently large batches and data-loader throughput. Cloud economics favor fewer well-sized files and autoscaled/spot compute when retries are safe.

11. Common Use Cases

Nightly features, weekly retraining, historical evaluation, large media preprocessing, embedding corpora, billing/reporting, periodic drift statistics, and bulk prediction exports.

12. Common Mistakes

- Reading all history for every run.
- Partitioning by a high-cardinality key or producing tiny files.
- Publishing partial results before validation.
- Assuming equal file count means equal work.
- Increasing workers while overwhelming a database/object store.
- No deterministic model/data version in prediction output.
- Choosing streaming when a 15-minute micro-batch meets requirements.

13. Edge Cases / Limitations

Late files, corrected historical partitions, skew, transient object-store listings, overlapping runs, GPU OOM from variable records, and a single poison record can break a batch. Batch latency cannot serve truly immediate decisions. Large backfills may violate current-run SLAs unless separately scheduled and throttled.

14. Variations

- Full batch: recompute entire dataset; simplest for small data.
- Incremental batch: process changed partitions; common in production.
- Micro-batch: very short bounded intervals; used by Spark Structured Streaming.
- MapReduce: explicit map/shuffle/reduce model; core interview foundation.
- Vectorized batch inference: groups records for efficient CPU/GPU model execution.

15. Related Topics

Airflow triggers batches, Spark distributes them, Parquet enables pruning, warehouses execute batch SQL, and lakehouse snapshots enable atomic/versioned publication. Streaming ML trades this simplicity for lower latency.

16. Interview Questions

1. **Batch vs streaming?** Batch processes bounded data with higher latency and simpler recovery; streaming continuously processes unbounded events.
2. **What determines batch wall time?** Critical-stage stragglers, available parallelism, I/O/shuffle, queues, and serial work.
3. **How do you retry safely?** Use deterministic partition outputs or transactional keyed writes.
4. **How do you choose partitions?** Align with filters/retry boundaries and target sufficiently large, balanced tasks/files.
5. **What is a straggler?** A slow task that delays its entire stage.
6. **How do you optimize repeated daily jobs?** Incremental reads, pruning, local combines, reuse/materialization, and right-sized resources.
7. **How do you publish output?** Stage, validate, then atomically commit a snapshot or replace scoped partitions.
8. **Why might more workers not help?** Serial work, skew, scheduler overhead, bandwidth limits, or source throttling.
9. **When is micro-batch enough?** When its interval plus processing meets the product latency SLO.
10. **How do you operate backfills?** Pin versions, parameterize ranges, isolate capacity, checkpoint, validate, and throttle publication.

17. Practice Tasks

- Code: parallelize deterministic scoring across Parquet files.
- Dataset project: nightly taxi-demand features and forecasts.
- Experiment: measure throughput vs partition/file size.
- Debugging: diagnose a long-tail stage from skewed partitions.
- Extension: add staged atomic publication and restart from failed partitions.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Batch recommender	Builds interactions, trains, scores all users	Spark, implicit feedback	Scalable ML workflow
Embedding backfill	Shards and embeds documents/images	PyTorch, Parquet	GPU/data throughput skills
Cost-aware batch runner	Benchmarks partitioning and autoscaling	Spark/cloud-like local cluster	Systems optimization

19. Quick Revision

- Key idea: bounded input enables efficient, reproducible parallel execution.
- Formula: $T_P \approx W/P + \text{overhead}$; stragglers limit stages.
- Metrics: throughput, SLA, skew, retries, cost.
- Trap: full rescans, tiny files, and partial publication.
- Interview one-liner: "Use batch whenever its latency satisfies the product; it is cheaper and easier to replay."

20. Final Cheat Sheet

Item	Summary
Input/output	Finite snapshot/partitions → committed result snapshot
Main steps	bound → partition → process → shuffle → validate → commit
Key tuning	file/partition size, workers, memory, batch interval
Metrics	duration, throughput, skew, retries, cost
Pros/cons	simple/replayable/efficient / delayed results
Best uses	training sets, backfills, batch scoring, aggregates

Data Quality Checks

1. Overview

Data quality checks are executable assertions and statistical monitors that determine whether data is fit for intended use. In ML, quality includes not only schema validity but point-in-time correctness, label integrity, feature distribution stability, train/serve parity, and freshness. Checks can block publication, quarantine records, or warn operators depending on impact and confidence.

2. Intuition

Unit tests ask whether code follows its rules; data checks ask whether today's changing inputs still satisfy theirs. A pipeline can execute perfectly and deliver an empty, duplicated, stale, or leaked training table.

3. Prerequisites

Schemas, descriptive statistics, probability, SQL, ML validation and leakage, pipeline stages, alerting, and business/domain rules.

4. Core Concepts

Dimension	Meaning / why	Example	Interview angle
Completeness	Required records/fields exist.	Prediction coverage > 99%.	Null vs absent row
Validity	Values satisfy type/range/domain.	age BETWEEN 0 AND 120.	Reject vs coerce
Uniqueness	Keys match promised grain.	One row/user/cutoff.	Duplicate diagnosis
Consistency	Related facts agree.	End time ≥ start time.	Cross-table reconciliation
Timeliness	Data is fresh enough.	Max event time within 15 min.	Freshness SLO
Accuracy	Values reflect reality/reference.	Currency conversion correct.	Hardest dimension to test
Distribution	Statistical behavior remains plausible.	PSI/KS/drift.	Drift vs corruption
Referential integrity	Foreign keys resolve.	Every order has user.	Orphan records

5. Algorithm / Working Process

1. Define contracts from consumer needs and table grain.
2. Add deterministic schema, null, range, uniqueness, and referential checks.
3. Add reconciliation between source, accepted, rejected, and published counts.
4. Establish baselines for volume, freshness, and distributions using clean history.
5. Set action: hard fail, quarantine, warning, or observe-only.
6. Emit metrics with dataset/version/partition context.
7. Investigate alerts through lineage, correct data, backfill, and update thresholds only with evidence.

6. Mathematical Foundation

Completeness and duplicate rate:

$$C_j = 1 - \frac{\#NULL_j}{N}, \quad D = 1 - \frac{\#distinct(keys)}{N}.$$

Population Stability Index across bins i is

$$PSI = \sum_i (p_i - q_i) \ln \frac{p_i}{q_i},$$

where p is reference and q is current; use smoothing for zero bins and treat thresholds as domain-dependent. A z-score volume alert is $z = (x - \mu)/\sigma$, but seasonal data needs day/hour-specific baselines. Total variation distance is $\frac{1}{2} \sum_i |p_i - q_i|$. Statistical significance is not business significance, especially at large N .

7. Practical Implementation

```
import pandas as pd

def validate_feature_table(df: pd.DataFrame, cutoff: pd.Timestamp) -> dict[str, float]:
    required = {"user_id", "feature_time", "age", "label"}
    missing = required - set(df.columns)
    if missing:
        raise AssertionError(f"missing columns: {sorted(missing)}")

    metrics = {
        "rows": float(len(df)),
        "user_null_rate": float(df["user_id"].isna().mean()),
        "duplicate_rate": float(df.duplicated(["user_id",
"feature_time"]).mean()),
        "age_invalid_rate": float((~df["age"].between(0, 120) &
df["age"].notna()).mean()),
        "positive_rate": float(df["label"].mean()),
    }
    failures = {
        "empty": len(df) == 0,
        "null user": metrics["user_null_rate"] > 0,
        "duplicate grain": metrics["duplicate_rate"] > 0,
        "invalid age": metrics["age_invalid_rate"] > 0.001,
        "future feature": bool((df["feature_time"] > cutoff).any()),
        "invalid label": not df["label"].dropna().isin([0, 1]).all(),
    }
    if failed := [name for name, bad in failures.items() if bad]:
        raise AssertionError(f"quality failures: {failed}; metrics={metrics}")
    return metrics
```

8. Code Explanation

Checks first establish required columns, then compute metrics even for properties that currently pass. Exact invariants such as unique grain and no future feature time block immediately. The age threshold allows a small invalid fraction only if the product policy permits quarantine; thresholds should be agreed with owners rather than copied blindly.

9. Training / Evaluation

Evaluate a quality system by incident detection rate, false-alert rate, time to detection, coverage of critical contracts, actionability, and escaped defects—not the raw number of checks. Calibrate statistical monitors on representative seasonal history and validate them against known incidents. Separately monitor model performance because valid data can still become less predictive.

10. Complexity and Cost

Schema/range/null checks scan relevant columns in $O(N)$. Exact uniqueness/distinct checks require $O(K)$ memory or distributed shuffle; approximate sketches reduce cost. Referential checks are joins. Distribution checks add histograms/quantiles. Sample expensive exploratory checks, but never sample away a hard security, key, or publication invariant without quantified risk.

11. Common Use Cases

Ingestion contracts, feature-table gates, label validation, model input monitoring, prediction output coverage, source-to-target reconciliation, schema-change detection, and privacy/compliance field checks.

12. Common Mistakes

- Only checking schema, not semantics or distributions.
- Static thresholds that alert every holiday or miss gradual drift.
- Auto-imputing corruption and hiding the incident.
- Blocking an entire pipeline for a non-critical noisy statistic.
- Alerting without owner, dataset/partition, observed value, threshold, and runbook.
- Calculating a baseline with future/test data or after known corruption.
- Equating correlation/drift with causation or model degradation.

13. Edge Cases / Limitations

New categories may be valid product growth, a stable distribution can still contain incorrect values, and rare-segment failures disappear in global averages. Labels arrive late and may change. Small samples make statistical tests noisy; huge samples flag negligible shifts. Accuracy often requires external ground truth that is unavailable.

14. Variations

- Contract tests: deterministic schema/constraint checks; essential.
- Reconciliation tests: compare counts/sums across stages; essential for finance-like data.
- Statistical anomaly detection: adaptive volume/distribution checks; useful but requires tuning.
- Metamorphic tests: verify relationships under transformations; research/project valuable.
- Shadow checks: observe a new rule before making it blocking; operationally useful.

15. Related Topics

ETL creates quality boundaries, Airflow controls blocking and alerting, feature pipelines require train/serve parity, warehouses expose profiling SQL, and streaming systems add event-time completeness and late-data metrics.

16. Interview Questions

1. **What are core data-quality dimensions?** Completeness, validity, uniqueness, consistency, timeliness, accuracy, and referential integrity.
2. **Hard check vs soft check?** A hard invariant blocks/quarantines; a soft statistical monitor warns because legitimate variation is possible.

3. **How do you detect schema drift?** Compare incoming physical/logical schema to a versioned contract with compatibility rules.
4. **How do you check feature leakage?** Assert source/event times do not exceed each training row's cutoff and audit lineage.
5. **What is PSI?** A binned divergence-like score comparing current and reference proportions; useful as a heuristic, not a universal verdict.
6. **How do you reduce alert fatigue?** Make checks consumer-impact-aware, seasonal, deduplicated, owned, and actionable.
7. **Can a dataset pass checks and still be wrong?** Yes; checks cover known invariants and proxies, while real-world accuracy may lack ground truth.
8. **Where should checks run?** At trust boundaries and before publication, with selected consumer-side monitoring too.
9. **How do you validate streaming completeness?** Track offsets, event-time lag, watermark, late/drop counts, and expected source reconciliation.
10. **What do you do after a failure?** Stop unsafe publication, preserve evidence, trace lineage, repair/replay, validate, and document root cause.

17. Practice Tasks

- Code: build checks for grain, ranges, nulls, and future timestamps.
- Dataset project: profile and validate a public credit dataset.
- Experiment: compare PSI, KS, and Jensen-Shannon divergence on synthetic drift.
- Debugging: investigate sudden null increase isolated to one producer version.
- Extension: create segment-level checks and seasonal thresholds.

18. Project Ideas

Project	What it does	Stack/data	Resume value
ML data contract gate	Validates and reports training-table quality	Python, SQL, dashboard	Shows preventive ML reliability
Drift laboratory	Injects shifts and compares detectors	NumPy/scipy	Statistical depth
Pipeline reconciliation	Tracks counts/sums/rejects across stages	Airflow/dbt/Spark	Operational data engineering

19. Quick Revision

- Key idea: validate fitness for a specific consumer, not vague "cleanliness."
- Formulas: null rate, duplicate rate, PSI/TV distance.
- Metrics: freshness, validity, uniqueness, alert precision, MTDD.
- Trap: green schema with leaked, stale, or semantically wrong data.
- Interview one-liner: "Quality checks should protect declared contracts at publication boundaries and report actionable evidence."

20. Final Cheat Sheet

Item	Summary
Input/output	Dataset + contract/baseline → pass, quarantine, warn, metrics
Main steps	profile → assert → compare → decide → alert → remediate
Key tuning	thresholds, segments, windows, severity, baseline
Metrics	null/duplicate/invalid rate, freshness, drift, coverage
Pros/cons	catches silent failures / cannot prove truth, alert tuning
Best uses	trust boundaries, feature/label gates, monitoring

Spark Basics

1. Overview

Apache Spark is a distributed computation engine for large-scale SQL, batch, streaming, and ML workloads. It builds a logical plan from lazy transformations, optimizes it, splits work into stages and tasks, and executes across a driver plus worker executors. ML teams use Spark for large joins, feature generation, training-data assembly, distributed inference, and historical backfills.

2. Intuition

Spark is a project manager: it turns a large job into partition-sized tasks, sends them to workers near data when possible, and combines results. It delays work until an action so it can reorganize the whole plan rather than blindly executing each line.

3. Prerequisites

Python/SQL, partitions, serialization, joins/aggregations, DAGs, memory/disk/network basics, and batch processing. DataFrame API knowledge is more important for placements than low-level RDD mastery.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Driver/executor	Driver plans; executors run tasks and store partitions.	Cluster application.	Driver OOM causes
DataFrame	Distributed typed-schema table with optimizable operations.	Parquet feature table.	DataFrame vs RDD
Lazy evaluation	Transformations build plans; actions trigger execution.	<code>select/filter</code> then <code>count</code> .	Why useful?
Partition	Unit of parallel data and task scheduling.	200 Parquet splits.	Too many/few partitions

Subtopic	Meaning / why	Example	Interview angle
Narrow dependency	Child partition needs few parent partitions.	<code>map</code> , <code>filter</code> .	Stage boundary
Wide dependency/shuffle	Records redistribute across workers.	<code>group-by</code> , <code>join</code> .	Expensive operation
Catalyst/Tungsten	Logical optimization and efficient physical execution.	Predicate pushdown/codegen.	Why DataFrames outperform Python loops
Cache/persist	Reuse computed partitions.	Repeated training analyses.	When caching hurts

5. Algorithm / Working Process

1. Driver constructs unresolved then analyzed logical plans.
2. Catalyst applies rule/cost optimizations such as projection/filter pushdown.
3. Spark chooses physical operators and partitions a job into stages at shuffle boundaries.
4. Scheduler launches one task per partition on executors.
5. Executors read, transform, shuffle, spill, and write results.
6. Failed tasks are recomputed from lineage; actions return results or persist output.

6. Mathematical Foundation

With partition sizes n_j , stage time is approximately

$$T_{stage} \approx \max_j (T_{read,j} + T_{cpu,j} + T_{shuffle,j} + T_{spill,j}).$$

Balanced partitions ideally have $n_j \approx N/P$; skew can be summarized by $\max(n_j)/\text{median}(n_j)$. A broadcast join avoids shuffling a fact table of size F by sending dimension D to P executors: network is roughly $P|D|$ instead of repartitioning $F + D$, provided D fits memory.

7. Practical Implementation

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("ml-features").getOrCreate()

events = (
    spark.read.schema(
        "user_id string, event_time timestamp, amount double, event_date date"
    )
    .parquet("data/events")
    .where(F.col("event_date") >= F.lit("2026-08-01")) # partition pruning
    .select("user_id", "event_time", "amount")         # column pruning
)

features = (
```

```
events.groupBy("user_id")
  .agg(
    F.count("*").alias("event_count"),
    F.sum("amount").alias("total_amount"),
    F.max("event_time").alias("last_event_time"),
  )
  .repartition(64, "user_id")
)

features.write.mode("overwrite").parquet("output/user_features")
features.explain("formatted")
```

8. Code Explanation

An explicit schema avoids a discovery pass and type surprises. Date filtering and column selection allow Parquet partition/column pruning. `groupBy` creates a shuffle because all rows for a user must meet. The final repartition controls downstream file/task distribution; 64 is workload-specific, not a universal value. `explain` exposes the physical plan and pushed filters.

9. Training / Evaluation

Spark is configured rather than trained. Evaluate job duration, input/output/shuffle bytes, task skew, spill, garbage collection, executor failures, CPU utilization, and cost. Validate output counts, grain, and distributions independently. For Spark ML, use leakage-safe splits and metrics appropriate to the model; do not assume distributed training improves a model that fits on one machine.

10. Complexity and Cost

Local row operations are $O(N/P)$ per worker. Shuffles add serialization, network, sort, and disk work around $O(N)$ data volume, sometimes sorting $O(n_j \log n_j)$. `collect()` can move $O(N)$ data to driver and crash it. Python UDFs add serialization and prevent some optimizations; built-in SQL functions or vectorized UDFs are preferable. Cluster startup makes Spark poor for tiny workloads.

11. Common Use Cases

Terabyte-scale feature aggregation, lakehouse ETL, large joins, log processing, offline recommendations, distributed preprocessing, batch inference, and Structured Streaming micro-batches.

12. Common Mistakes

- Calling `collect()/toPandas()` on unbounded data.
- Using Python UDFs when built-ins exist.
- Caching one-use or oversized DataFrames.
- Repartitioning repeatedly or using `coalesce(1)` for a single output file.
- Ignoring skew and small files.
- Treating executor memory increase as the only cure for spill.
- Relying on inferred schemas or `SELECT *`.

13. Edge Cases / Limitations

Hot keys, nested skew, non-deterministic UDFs, executor loss, speculative duplicate task execution, and object-store commit behavior affect correctness/performance. Spark is not an OLTP database and does not offer low-latency per-row access. Small datasets may run faster in DuckDB/pandas/SQL warehouses.

14. Variations

- Spark SQL/DataFrames: default choice; high placement importance.
- RDD API: fine-grained unstructured control; know conceptually, use less often.
- Structured Streaming: incremental DataFrame execution with checkpoints/state.
- pandas API on Spark: migration convenience with semantic/performance caveats.
- Spark MLlib: scalable classical algorithms; useful when data genuinely exceeds one machine.

15. Related Topics

Parquet supplies pushdown; lakehouses add transactional metadata; distributed internals explain stages, shuffles, lineage, and faults; Airflow submits Spark applications; feature pipelines use Spark offline and another store online.

16. Interview Questions

1. **Why is Spark lazy?** It can optimize an entire plan and avoid unused work before an action executes.
2. **Transformation vs action?** Transformations return a new logical dataset; actions trigger execution or return/write results.
3. **Narrow vs wide transformation?** Narrow dependencies avoid global redistribution; wide dependencies require shuffle and new stages.
4. **What causes a shuffle?** A change in partitioning requirements, commonly joins, group-bys, distinct, repartition, and global sort.
5. **DataFrame vs RDD?** DataFrames expose schema/expressions to Catalyst; RDDs provide lower-level objects with fewer optimizations.
6. **Repartition vs coalesce?** Repartition shuffles to increase/decrease/balance; coalesce usually reduces partitions without full shuffle.
7. **When should you cache?** When expensive computed data is reused and fits an appropriate storage level.
8. **How do you diagnose skew?** Inspect task-duration/input/shuffle distributions and key frequencies; one/few tasks dominate.
9. **Why avoid Python UDFs?** Serialization overhead and optimizer/code-generation barriers; use built-ins when possible.
10. **What happens after executor task failure?** Spark retries and recomputes lost partitions from lineage/shuffle outputs, subject to retry limits.

17. Practice Tasks

- Code: aggregate partitioned Parquet without UDFs.
- Dataset project: build taxi-demand features in Spark.
- Experiment: compare broadcast and sort-merge joins using `explain`.
- Debugging: fix driver OOM caused by `collect`.
- Extension: salt a synthetic hot key and compare task tails.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Spark feature factory	Builds multi-window user features	PySpark, Parquet	Core data/ML engineering
Skew benchmark	Visualizes and repairs hot-key stages	PySpark, synthetic data	Distributed debugging
Batch inference job	Applies model artifact at scale with validation	Spark, MLflow/ONNX	Production ML serving path

19. Quick Revision

- Key idea: lazy DataFrame plan becomes partition tasks separated by shuffles.
- Formula: slowest partition controls stage time.
- Metrics: shuffle/spill, skew, task duration, executor/driver memory.
- Trap: `collect`, Python UDFs, blind caching/repartitioning.
- Interview one-liner: "Spark performance is mostly data layout, partition balance, and avoiding unnecessary shuffles."

20. Final Cheat Sheet

Item	Summary
Input/output	Distributed files/tables → distributed result
Main steps	logical plan → optimize → stages → partition tasks → commit
Key tuning	partitions, join strategy, cache, memory, file size
Metrics	runtime, shuffle, spill, skew, GC, failures
Pros/cons	scalable/fault-tolerant SQL / startup, shuffle, operational cost
Best uses	large ETL, joins, features, backfills, batch inference

Airflow Basics

1. Overview

Apache Airflow is a workflow orchestrator for defining, scheduling, and monitoring primarily batch workflows as Python-authored DAGs. It coordinates external computation; it should not itself process large datasets. ML uses include scheduled feature materialization, validation, training, evaluation, model registration, batch inference, and monitoring.

2. Intuition

Airflow is an airport control tower: it decides when jobs may start, respects dependencies, records outcomes, retries, and alerts. The planes—Spark, SQL, containers, APIs—perform the actual work.

3. Prerequisites

Python functions, DAGs, scheduling/cron, idempotency, ETL, external storage/compute, credentials, and retry/failure concepts.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
DAG	Workflow definition and dependency graph.	ingest → validate → train.	Parse-time behavior
Task/operator	One orchestration unit using an operator.	Submit Spark job.	Task granularity
Logical date/data interval	Interval a run represents, not wall-clock start.	Daily run processes previous day.	Common date bug
Scheduler/executor	Scheduler selects tasks; executor runs/dispatches them.	Celery/Kubernetes/local.	Scaling architecture
XCom	Small task metadata exchange.	Published table URI.	Why not pass DataFrames?
Retry/timeout	Bounded recovery from transient failure.	API retry with backoff.	Idempotency requirement
Catchup/backfill	Create historical logical runs.	Recompute 30 days.	Resource control
Sensor/deferrable task	Wait for external condition efficiently.	File/partition ready.	Worker-slot usage

5. Algorithm / Working Process

1. Scheduler repeatedly parses DAG definitions and creates eligible DAG runs.
2. For each run, it evaluates task dependencies, trigger rules, pools, concurrency, and previous state.
3. Ready task instances are queued to the executor.
4. Workers execute operators that call external systems.
5. Metadata DB records state, retries, timing, and small XCom values.
6. Failures retry or alert; downstream tasks run only when trigger rules allow.
7. Operators publish versioned results and quality tasks gate consumers.

6. Mathematical Foundation

The DAG's earliest completion follows critical-path length

$$T_{critical} = \max_{p \in paths} \sum_{i \in p} T_i.$$

With retry count r and independent attempt failure probability p , probability all $r + 1$ attempts fail is p^{r+1} , but correlated or deterministic failures invalidate this optimistic model. Exponential backoff can be expressed $d_k = \min(d_{max}, d_0 2^k)$.

7. Practical Implementation

```
from datetime import datetime, timedelta
from airflow.decorators import dag, task

@dag(
    schedule="0 3 * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={"retries": 2, "retry_delay": timedelta(minutes=5)},
    tags=["ml", "features"],
)
def daily_features():
    @task
    def build(data_interval_start=None, data_interval_end=None) -> str:
        # Submit SQL/Spark work; return only small metadata through XCom.
        partition = data_interval_start.date().isoformat()
        run_feature_job(partition=partition) # idempotently replaces this
        partition
        return f"features/event_date={partition}"

    @task
    def validate(uri: str) -> None:
        metrics = validate_partition(uri)
        if metrics["rows"] == 0 or metrics["duplicate_rate"] > 0:
            raise ValueError(f"invalid feature partition: {metrics}")

    validate(build())

daily_features()
```

8. Code Explanation

The DAG uses the data interval to choose a deterministic partition; it does not use `datetime.now()`, so reruns target the same data. The build task delegates computation and returns only a URI. Validation blocks downstream success. `catchup=False` is appropriate only when automatic historical creation is not wanted; explicit backfills can still be run.

9. Training / Evaluation

Airflow is not trained. Evaluate DAG success and SLA/SLO attainment, queue time, task duration, retry/failure rates, scheduler health, pool saturation, freshness, and validation results. For model workflows, record data/model/code versions and evaluation gates. Test DAG import, task logic outside Airflow where possible, and one end-to-end run in an isolated environment.

10. Complexity and Cost

Scheduler overhead grows with DAG/task count and parse frequency. Dynamic task mapping over enormous item counts overloads metadata; submit coarse distributed jobs instead. Long polling sensors waste slots unless rescheduled/deferrable. Airflow workers need enough capacity for orchestration calls, not the memory required by the underlying dataset.

11. Common Use Cases

Daily ETL, warehouse transformations, Spark submission, retraining pipelines, feature refreshes, validation/reporting, batch prediction, and historical backfills.

12. Common Mistakes

- Heavy network/database work at DAG import time.
- Passing DataFrames/files through XCom.
- Using wall-clock “now” instead of logical intervals.
- Non-idempotent tasks combined with automatic retries.
- Thousands of tiny tasks or sensors occupying workers.
- Storing credentials in code or logs.
- Treating Airflow as a streaming/event-processing engine.

13. Edge Cases / Limitations

DAG code changes can affect historical reruns; pin job artifacts/config where reproducibility matters. Overlapping runs may write the same partitions. Scheduler/metadata DB availability affects orchestration, while external jobs may continue. Airflow has minute-scale scheduling and is unsuitable for per-event millisecond processing.

14. Variations

- Operators/hooks: integrations with SQL, cloud, containers, Spark.
- TaskFlow API: Pythonic task definitions and dependency inference; common modern style.
- Celery/Kubernetes executors: distributed or pod-isolated execution.
- Datasets/assets scheduling: trigger on updated data assets rather than only time.
- Managed Airflow: reduces control-plane operations but not DAG/data design.

15. Related Topics

Airflow orchestrates ETL and batch pipelines, invokes Spark/warehouse queries, gates on quality, and can publish feature/model versions. Kafka/stream processors run continuously and usually are deployed/monitored rather than event-by-event scheduled by Airflow.

16. Interview Questions

1. **What is Airflow?** A metadata-backed scheduler/orchestrator for dependency-aware workflows, mainly batch.
2. **DAG vs task instance?** DAG is the definition; a task instance is one task for one DAG run/logical interval.

3. **What is logical date?** The data interval represented by a run, not necessarily its actual execution time.
4. **Why must tasks be idempotent?** Retries and manual reruns are normal and must not duplicate effects.
5. **What belongs in XCom?** Small control metadata such as URIs, IDs, and counts—not bulk data.
6. **How do you wait efficiently?** Deferrable operators or reschedule-mode sensors release worker capacity.
7. **What is catchup?** Automatic creation of past scheduled runs from the start date.
8. **How do you limit backfill impact?** Pools, concurrency limits, separate queues, priority, and bounded date ranges.
9. **Where should large computation run?** An external engine/service submitted by an Airflow task.
10. **How do you prevent overlapping-run corruption?** Non-overlapping intervals, max-active-run controls, transactional outputs, or idempotent partition writes.

17. Practice Tasks

- Code: define ingest → validate → feature DAG using logical dates.
- Dataset project: schedule retraining and batch prediction for a public dataset.
- Experiment: compare polling and deferrable waits.
- Debugging: fix an off-by-one-day partition caused by wall-clock time.
- Extension: add backfill parameters, pools, alerts, and model-evaluation gate.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Training orchestrator	Builds features, trains, validates, registers	Airflow, Spark, MLflow	End-to-end MLOps
Quality-gated ETL	Publishes only validated partitions	Airflow, SQL, Parquet	Reliability patterns
RAG refresh DAG	Detects documents, embeds, atomically updates index	Airflow, vector store	GenAI operations

19. Quick Revision

- Key idea: Airflow coordinates work and state; external systems process data.
- Formula: completion follows DAG critical path.
- Metrics: queue/duration/retry/SLA plus data freshness and quality.
- Trap: `now()`, XCom data payloads, non-idempotent retries.
- Interview one-liner: “Airflow is an orchestrator, not a data-processing engine.”

20. Final Cheat Sheet

Item	Summary
Input/output	Schedule/events + DAG → external job executions/results
Main steps	parse → create run → schedule ready tasks → execute → record/alert
Key tuning	concurrency, pools, retries, timeouts, catchup

Item	Summary
Metrics	queue time, duration, retries, failures, SLA/freshness
Pros/cons	visible dependency/retry control / metadata and scheduling overhead
Best uses	batch ETL, training, scoring, validation, backfills

Kafka Basics

1. Overview

Apache Kafka is a distributed, durable event-log platform. Producers append records to partitioned topics; brokers persist and replicate them; consumers read ordered partition logs using offsets. Kafka decouples producers from feature processors, monitoring, storage sinks, and real-time inference systems while retaining events for replay.

2. Intuition

Kafka is a replicated notebook with numbered lines. Writers append; each reader remembers its own line number. Multiple teams can read independently, and a reader can rewind if retained history still exists.

3. Prerequisites

Networking, serialization/schema, partitions, hashing, replication, consumer/producer basics, batch vs streaming, idempotency, and distributed failure.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Topic/partition	Named log split into ordered shards.	<code>transactions</code> with 24 partitions.	Ordering scope
Key	Chooses partition and groups related records.	<code>account_id</code> .	Hot-key risk
Offset	Position within one partition.	Consumer checkpoint.	Offset vs event ID
Consumer group	One active consumer per partition within a group.	Scoring service replicas.	Max parallelism
Replication/leader	Leader handles partition I/O; followers replicate.	RF=3.	Failure durability
Delivery semantics	At-most-once, at-least-once, exactly-once processing effect.	Deduplicated sink.	End-to-end semantics
Retention/compaction	Keep by time/size or latest value per key.	User profile changelog.	Replay/history

Subtopic	Meaning / why	Example	Interview angle
Rebalance	Reassign partitions as membership changes.	Replica added.	Processing pause/state movement

5. Algorithm / Working Process

1. Producer serializes record, chooses partition by key or strategy, batches/compresses, and sends to leader.
2. Broker appends to partition log and replicates to in-sync replicas.
3. Acknowledgment policy determines when the send succeeds.
4. Consumer group coordinator assigns partitions.
5. Consumers poll batches in partition order, process them, and commit offsets.
6. On failure/rebalance, another member resumes from committed offsets; uncommitted records may be reprocessed.

6. Mathematical Foundation

If arrival rate is λ records/s and sustainable consumption is μ , lag changes approximately as

$$\frac{dL}{dt} = \lambda - \mu.$$

Stable processing needs long-run $\mu > \lambda$ with headroom. With P partitions, a consumer group has at most P actively consuming members. Replication factor r multiplies broker storage/network roughly by r . For uniformly hashed keys, expected load is N/P , but real key skew determines the maximum.

7. Practical Implementation

```
import json
from confluent_kafka import Producer, Consumer

producer = Producer({"bootstrap.servers": "localhost:9092", "enable.idempotence":
True})
producer.produce(
    "transactions",
    key=b"account-42",
    value=json.dumps({"event_id": "e-100", "amount": 99.5}).encode(),
)
producer.flush()

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "fraud-features-v1",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
})
consumer.subscribe(["transactions"])
try:
```

```
while message := consumer.poll(1.0):
    if message.error():
        raise RuntimeError(message.error())
    event = json.loads(message.value())
    upsert_feature(event["event_id"], event) # sink deduplicates by stable ID
    consumer.commit(message=message, asynchronous=False)
finally:
    consumer.close()
```

8. Code Explanation

The record key keeps one account's ordered events in one partition. Idempotent production prevents duplicates from producer retries within its guarantees. Manual commit occurs after the sink effect; a crash between sink write and commit reprocesses the event, so the sink's upsert/dedup key makes the end-to-end effect idempotent. Production code handles timeouts, malformed messages, pause/backpressure, and dead-letter policy.

9. Training / Evaluation

Kafka has configuration, not model training. Evaluate produce/consume throughput, end-to-end/event-time latency, consumer lag, under-replicated partitions, ISR changes, broker disk/network, rebalance rate, error/retry rate, and duplicate/drop outcomes. Load-test record-size and key distributions, not only uniform synthetic traffic.

10. Complexity and Cost

Append and sequential reads are effectively $O(1)$ per record plus serialization/network. Storage is approximately $\text{ingress bytes} \times \text{retention duration} \times \text{replication factor}$, adjusted for compression. More partitions increase parallelism but also metadata, files, connections, rebalances, and ordering fragmentation. Small unbatched records waste network and broker overhead.

11. Common Use Cases

Event ingestion, CDC transport, real-time feature updates, fraud signals, clickstream analytics, model prediction/feedback logs, online monitoring, and decoupled lake/warehouse sinks.

12. Common Mistakes

- Claiming global ordering; Kafka orders only within a partition.
- Committing before processing and losing effects on crash.
- Processing then committing to a non-idempotent sink and duplicating effects.
- Choosing a constant/hot key or no key when entity order matters.
- No versioned schema or compatibility policy.
- Treating Kafka as permanent object storage/database query engine.
- Excessive partitions or very large messages.

13. Edge Cases / Limitations

Poison messages can block a partition; use bounded retries plus quarantine with observability. Rebalances interrupt processing. Clock skew makes producer timestamps unreliable for some event-time use. Retention may expire before replay. Cross-topic/external-sink exactly-once behavior requires transactions and compatible processing/sinks; Kafka alone cannot guarantee arbitrary side effects exactly once.

14. Variations

- Kafka Streams: embedded stream-processing library with local state and exactly-once options.
- Kafka Connect: standardized source/sink connectors; important for practical ingestion.
- Log compaction: retains latest record per key plus tombstones; useful for state changelogs.
- Schema Registry with Avro/Protobuf/JSON Schema: manages compatibility.
- Managed Kafka: reduces broker operations, retains data/consumer design responsibilities.

15. Related Topics

Streaming ML consumes feature/prediction events; Spark Structured Streaming processes Kafka inputs; lakehouse sinks retain history; data pipelines monitor lag and schema; distributed internals explain replication, consensus-like coordination, and failure semantics.

16. Interview Questions

1. **What is Kafka?** A replicated partitioned append-only log for durable event streaming.
2. **What ordering does Kafka guarantee?** Record order within a partition, not across a topic.
3. **Why use a key?** Stable partitioning and per-entity order; poor keys can create skew.
4. **What is an offset?** A record position within one topic partition, tracked per consumer group.
5. **How does a consumer group scale?** Partitions are divided among members; active members cannot exceed partition count.
6. **At-least-once meaning?** Records are not intentionally lost but may be reprocessed, so effects need idempotency/deduplication.
7. **When do you commit offsets?** After the corresponding effect is durably complete for at-least-once processing.
8. **Retention vs compaction?** Retention removes old segments by time/size; compaction preserves latest values per key while retaining log behavior.
9. **What is consumer lag?** Difference between latest partition offset and consumer's processed/committed position.
10. **What happens on broker failure?** An eligible in-sync replica is elected leader; durability depends on replication and acknowledgment settings.

17. Practice Tasks

- Code: produce keyed JSON events and consume with manual commits.
- Dataset project: stream synthetic transactions into fraud-window features.
- Experiment: measure batching/compression and partition-count throughput.
- Debugging: explain duplicates after consumer crash and rebalance.
- Extension: add schema validation, quarantine topic, and idempotent sink.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Fraud event backbone	Keyed events → window features → alerts	Kafka, Flink/Spark	Streaming ML architecture
CDC lake sink	Captures DB changes into lakehouse tables	Kafka Connect, Debezium, Iceberg	Modern data platform
Prediction feedback bus	Joins predictions with delayed outcomes	Kafka, Python	Online monitoring design

19. Quick Revision

- Key idea: replicated partitioned logs decouple producers and replayable consumers.
- Formula: lag grows at $\lambda - \mu$; group parallelism $\leq$ partitions.
- Metrics: lag, throughput, latency, ISR, rebalance, disk.
- Trap: global-order and end-to-end exactly-once claims.
- Interview one-liner: "Kafka gives ordered replay within partitions; consumers still design idempotent effects."

20. Final Cheat Sheet

Item	Summary
Input/output	Producer records → retained partition logs → consumer records
Main steps	serialize → partition → append/replicate → poll → process → commit
Key tuning	partitions, key, acks, batch/compression, retention
Metrics	throughput, lag, latency, ISR, rebalance
Pros/cons	durable/replayable/decoupled / operational and semantic complexity
Best uses	events, CDC, feature updates, feedback, stream inputs

Data Warehouses

1. Overview

A data warehouse is a governed analytical system optimized for large scans, joins, and aggregates across integrated historical data. Modern cloud warehouses separate or elastically manage storage and compute, use columnar storage, and provide SQL, transactions, access control, and workload management. ML teams use warehouses for cohort discovery, feature/label marts, BI, experimentation, governance, and batch predictions.

2. Intuition

An operational database is a cashier optimized for individual transactions; a warehouse is the headquarters analyst optimized to scan years of receipts. Sending headquarters-style questions to every cashier disrupts sales.

3. Prerequisites

SQL, OLTP vs OLAP, schemas/keys, columnar formats, partitioning/clustering, dimensions/facts, ETL/ELT, security, and cost basics.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Fact table	Events/measures at explicit grain.	One row per order line.	Declare grain first
Dimension table	Descriptive entity attributes.	Customer/product dimension.	Slowly changing dimensions
Star schema	Facts linked directly to denormalized dimensions.	Sales mart.	Analytics performance
Columnar execution	Scan/compress selected columns efficiently.	Query amount/date only.	OLAP vs OLTP
Partition/cluster	Physical pruning/locality.	Date partition, user clustering.	Cardinality choice
Materialized view	Persisted query result refreshed by engine.	Daily feature aggregate.	Freshness vs cost
Workload management	Isolate/queue resource consumers.	BI vs ML backfill pools.	Concurrency
Governance	Catalog, lineage, masking, row/column policy.	PII-controlled features.	Least privilege

5. Algorithm / Working Process

1. Ingest raw data through batch/CDC.
2. Standardize into staging and conformed dimensions/facts.
3. Build consumer-facing marts with declared grain and tests.
4. Query optimizer prunes partitions/columns, estimates cardinalities, and chooses distributed joins/aggregates.
5. Compute workers scan compressed data and exchange intermediate results.
6. Results may be cached/materialized and governed for consumers.
7. Usage, freshness, lineage, and spend are monitored.

6. Mathematical Foundation

Warehouse scan cost is often approximately

$$Cost \propto Bytes_{scanned} = Bytes_{table} \times f_{partitions} \times f_{columns} \times f_{blocks}.$$

Star-schema join output follows key frequency/cardinality rules from joins. A slowly changing dimension Type 2 lookup selects the row satisfying

$$valid_from \leq t_{event} < valid_to.$$

Compression and column pruning lower I/O; queueing and concurrency determine latency beyond query compute.

7. Practical Implementation

```
CREATE TABLE feature_mart AS
SELECT
  c.user_id,
  c.cutoff_date,
  COUNTIF(e.event_time > c.cutoff_date - INTERVAL '30' DAY
          AND e.event_time <= c.cutoff_date) AS events_30d,
  SUM(CASE WHEN e.event_time > c.cutoff_date - INTERVAL '30' DAY
            AND e.event_time <= c.cutoff_date THEN e.amount ELSE 0 END) AS
  spend_30d
FROM prediction_cutoffs c
LEFT JOIN fact_events e
  ON e.user_id = c.user_id
  AND e.event_date BETWEEN c.cutoff_date - INTERVAL '30' DAY AND c.cutoff_date
GROUP BY c.user_id, c.cutoff_date;

-- Quality gate: should return zero rows.
SELECT user_id, cutoff_date, COUNT(*) AS n
FROM feature_mart
GROUP BY user_id, cutoff_date
HAVING COUNT(*) > 1;
```

8. Code Explanation

The event-date condition enables partition pruning while exact timestamp conditions enforce feature semantics. The cohort remains the left side so inactive users survive. The final check protects one row per entity/cutoff. SQL syntax such as `COUNTIF` and interval notation varies by warehouse.

9. Training / Evaluation

Evaluate query latency, concurrency, scanned bytes, credits/cost, cache dependence, spill, freshness, and SLO. Data marts require grain, reconciliation, schema, and access tests. ML datasets require snapshot/version, point-in-time checks, train/serve parity, and downstream validation—not only fast SQL.

10. Complexity and Cost

Analytical scans are $O(N)$ in selected data; distributed joins/group-bys incur shuffle. Columnar storage and pruning reduce constants/data volume dramatically. Denormalization may reduce joins but increases storage and update complexity. Poor filters, `SELECT *`, repeated CTE scans, and uncontrolled backfills can create large cloud bills. CPU is abstracted as warehouse slots/credits; GPUs are generally outside standard warehouses.

11. Common Use Cases

Enterprise reporting, customer 360, experimentation, audit history, feature/label marts, training-data extracts, model monitoring, and reverse ETL outputs.

12. Common Mistakes

- Fact table with undefined/mixed grain.
- Over-normalizing analytics or denormalizing without ownership.
- Partitioning by high-cardinality keys.
- Joining current dimension state to historical facts and rewriting history.
- Unbounded queries and no spend quotas.
- Using warehouse as millisecond online feature store.
- Copying sensitive data into ML marts without purpose-based access.

13. Edge Cases / Limitations

Late facts and dimension corrections require restatement policies. Concurrency, cold caches, maintenance, and provider quotas affect latency. SQL UDF/model features may not match application serving code. Warehouses prioritize analytical throughput, not low-latency transactional writes or per-request serving.

14. Variations

- Kimball dimensional modeling: stars/facts/dimensions; placement-important.
- Data Vault: historized hubs/links/satellites; enterprise-specific.
- Serverless warehouse: elastic pay-per-scan/compute; operationally simple but cost needs guardrails.
- Warehouse-native ML: convenient SQL training/scoring; limited model/runtime portability.
- Lakehouse: open-file/object-store data plus warehouse-like management; covered separately.

15. Related Topics

ETL/ELT populate warehouses, SQL queries them, Spark may complement them for custom processing, lakehouses converge on warehouse capabilities, and feature pipelines use warehouse data for offline computation.

16. Interview Questions

1. **OLTP vs warehouse?** OLTP handles many small consistent transactions; warehouses optimize large analytical scans/aggregates.
2. **Fact vs dimension?** Facts hold measurable events at a grain; dimensions describe entities and context.
3. **Star vs snowflake schema?** Star denormalizes dimensions for simpler/faster analytics; snowflake normalizes them for reuse/integrity.
4. **What is SCD Type 2?** Preserve attribute history with validity intervals and new rows per change.
5. **Partitioning vs clustering?** Partitioning creates coarse pruning units; clustering/sorting improves locality/block pruning within them.
6. **Why columnar storage?** Analytics reads few columns over many rows, enabling projection and compression.
7. **How do you reduce warehouse cost?** Prune partitions/columns, materialize reused work, right-size compute, quotas, and monitor scans.

8. **Can a warehouse be a feature store?** It can be the offline store, but usually not the low-latency online serving layer.
9. **How do you model historical features?** Join facts to dimension/feature state valid at the prediction time, not current state.
10. **What is workload isolation?** Separate resources/queues so BI, ETL, and ML backfills do not starve one another.

17. Practice Tasks

- Code: design a star schema and write a point-in-time feature mart query.
- Dataset project: warehouse a retail dataset and build churn labels.
- Experiment: measure scans with/without partition filters and projection.
- Debugging: fix historical facts joined to current customer tier.
- Extension: add SCD Type 2 and row/column access controls.

18. Project Ideas

Project	What it does	Stack/data	Resume value
ML-ready retail warehouse	Facts/dimensions, tests, feature mart	DuckDB/dbt; retail data	Modeling + SQL + ML
Experiment warehouse	Ingests exposures/outcomes and computes uplift	SQL, dashboard	Product analytics depth
Cost observatory	Profiles query scans and optimization gains	Warehouse/DuckDB	FinOps and performance

19. Quick Revision

- Key idea: governed columnar analytical storage around facts, dimensions, and marts.
- Formula: scan cost tracks selected partitions × columns × blocks.
- Metrics: scanned bytes, latency, concurrency, freshness, cost.
- Trap: undefined grain and current-state joins to historical facts.
- Interview one-liner: “A warehouse optimizes governed historical analytics; facts have an explicit grain and dimensions provide context.”

20. Final Cheat Sheet

Item	Summary
Input/output	Integrated historical sources → SQL marts/results
Main steps	ingest → stage → conform facts/dimensions → mart → govern/query
Key tuning	partitions, clustering, materialization, compute size
Metrics	scan bytes, latency, credits, freshness, concurrency
Pros/cons	governed fast analytics / cost, vendor/dialect, serving latency
Best uses	BI, cohorts, features/labels, experiments, monitoring

Feature Pipelines

1. Overview

A feature pipeline converts raw data into model-ready values for offline training and online/batch inference. It defines feature semantics, transformations, time boundaries, materialization, storage, freshness, versioning, and monitoring. Its hardest requirements are point-in-time correctness and training-serving consistency: the model must see features computed with information available at prediction time and with equivalent logic in both environments.

2. Intuition

A feature is a question with a timestamp: "How much had this user spent in the previous 30 days as of 10:00?" Removing "as of" changes a valid historical feature into leaked hindsight. A feature pipeline is the machinery that asks the same question reliably during training and serving.

3. Prerequisites

ML preprocessing, train/validation/test splits, leakage, SQL joins/aggregations, event time, batch/stream processing, storage, serialization, and model-serving latency.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Feature definition	Named transform with entity, type, source, owner, and time semantics.	<code>txn_count_7d</code> .	What makes a feature reusable?
Event vs processing time	When fact occurred vs system observed it.	Delayed transaction.	Historical correctness
Point-in-time join	Retrieve state available at row cutoff.	Latest profile before prediction.	Prevent leakage
Offline store	Historical feature values for training/batch.	Parquet/warehouse.	Scale/reproducibility
Online store	Latest values for low-latency lookup.	Redis/Cassandra.	Freshness/latency
Materialization	Compute and publish feature values.	Hourly offline-to-online sync.	Backfill and idempotency
Feature registry	Metadata/discovery/version/lineage.	Schema and owner.	Store vs registry
Train/serve skew	Different logic/data/freshness creates mismatched values.	Timezone or default differs.	Detection/remediation

5. Algorithm / Working Process

1. Define entity key, event timestamp, feature formula/window, default, type, freshness SLO, and owner.
2. Validate and deduplicate source events.
3. For offline data, create prediction cutoffs and compute/join only records with event time at or before each cutoff.
4. For online serving, incrementally update or materialize the same logical feature definition.
5. Store offline history and online latest values with timestamps/version.
6. Fetch a feature vector, apply fitted preprocessing, and log feature/version with predictions.
7. Monitor freshness, null/default rate, parity, drift, and downstream quality; replay/backfill safely.

6. Mathematical Foundation

A windowed feature for entity u at cutoff t is

$$f_{u,t}^{(W)} = A(\{x_i : u_i = u, t - W < t_i \leq t\}),$$

where A may be count, sum, or a mergeable state. Recency is $r_{u,t} = t - \max\{t_i : t_i \leq t\}$. An exponentially decayed feature is

$$f_{u,t} = \sum_{i:t_i \leq t} x_i e^{-\lambda(t-t_i)}, \quad t_{1/2} = \ln 2 / \lambda.$$

Parity error for sampled offline/online values can be measured as mean absolute difference or exact-match rate, with timestamp/tolerance semantics for eventually consistent updates.

7. Practical Implementation

```
import pandas as pd

def point_in_time_count(events: pd.DataFrame, cutoffs: pd.DataFrame, days: int) ->
pd.DataFrame:
    """Clear reference implementation for correctness; use SQL/Spark at scale."""
    pairs = cutoffs.merge(events, on="user_id", how="left",
validate="one_to_many")
    start = pairs["cutoff_time"] - pd.to_timedelta(days, unit="D")
    eligible = pairs[
        pairs["event_time"].notna()
        & (pairs["event_time"] > start)
        & (pairs["event_time"] <= pairs["cutoff_time"])
    ]
    counts = (
        eligible.groupby(["user_id", "cutoff_time"], as_index=False)
        .size()
        .rename(columns={"size": f"event_count_{days}d"})
    )
    result = cutoffs.merge(counts, on=["user_id", "cutoff_time"], how="left")
    result[f"event_count_{days}d"] =
result[f"event_count_{days}d"].fillna(0).astype("int64")
    assert not result.duplicated(["user_id", "cutoff_time"]).any()
    return result
```

8. Code Explanation

The cutoffs table establishes training row grain. Candidate events are joined by entity, then bounded by both sides of the lookback window. Users without events receive a documented zero. This reference can test optimized SQL/stream implementations on sampled cases; for large histories, use time-range joins or pre-aggregated buckets rather than a broad Cartesian entity join.

9. Training / Evaluation

Fit imputers, scalers, vocabularies, encoders, PCA, and feature selection using training data only; persist them with the model. Evaluate features by coverage, freshness, stability, leakage audits, offline/online parity, incremental model lift, robustness across time/segments, and inference availability. Use time-based validation when deployment predicts future outcomes. Ablation tests estimate whether a feature family adds value.

10. Complexity and Cost

Naive entity-time joins can approach $O(C \times E_u)$ per entity and explode. Window aggregation with ordered events is $O(N)$ plus state; bucketed batch features trade storage for reuse. Online fetch cost is roughly number of entity keys/store calls, so multi-get and colocated feature groups matter. Offline storage scales with entities $\times$ timestamps $\times$ feature versions; online memory holds latest state and metadata.

11. Common Use Cases

Fraud velocity, recommendation histories, churn engagement, ad ranking, credit risk, sensor summaries, user/item embeddings, document metadata, and real-time personalization.

12. Common Mistakes

- Joining latest/current features into historical training data.
- Computing preprocessing on all data before splitting.
- Different default values, time zones, or formulas online and offline.
- Missing feature timestamps/freshness and silently serving stale values.
- Reusing a feature name after changing its semantics.
- Materializing high-cardinality, low-value features without cost controls.
- Random validation split for temporal prediction.

13. Edge Cases / Limitations

Late/out-of-order events can revise historical truth. Entities may be new, deleted, merged, or have no history. Online stores are eventually consistent and may partially fail. Backfilled features can differ from values actually served historically. Some transformations—global target encoding or expensive embeddings—need special leakage/version handling.

14. Variations

- Batch feature pipeline: simplest for daily/hourly features; placement essential.
- Streaming feature pipeline: stateful low-latency updates; advanced systems relevance.
- Request-time/on-demand feature: combines request context and stored features; necessary for context-dependent ranking.

- Feature store: registry plus offline/online storage and retrieval/materialization services; valuable but not mandatory for small projects.
- Learned features/embeddings: require model artifact and version alongside data transform.

15. Related Topics

Aggregations define feature values; joins enforce point-in-time retrieval; warehouses/lakehouses are offline stores; Kafka and streaming update online state; data-quality checks detect freshness/parity; MLOps versions preprocessing and models together.

16. Interview Questions

1. **What is point-in-time correctness?** Every feature value uses only information available at that training row's historical prediction cutoff.
2. **What is train/serve skew?** The model sees different feature semantics, distributions, defaults, or freshness between training and inference.
3. **Offline vs online store?** Offline holds scalable history for training; online serves latest values at low latency.
4. **How do you prevent leakage?** Explicit cutoffs, time-bounded sources/joins, training-only fitted transforms, lineage, and temporal tests.
5. **What metadata belongs with a feature?** Name/version, entity, type, formula, source, event timestamp, owner, freshness, default, and lineage.
6. **How do you handle late events?** Define allowed lateness/revision policy, update state, backfill affected offline partitions, and measure corrections.
7. **How do you test parity?** Compute offline and online values for identical entity/cutoff samples and compare within defined consistency tolerances.
8. **Should all features be precomputed?** No; precompute expensive reusable features, calculate request-context features on demand when needed.
9. **How do you handle missing online features?** Typed defaults/fallback model or fail policy, with missingness and freshness logged and monitored.
10. **Why version features?** Semantic changes must not silently alter training or serving behavior for existing model versions.

17. Practice Tasks

- Code: construct leakage-safe 7/30-day features for multiple cutoffs.
- Dataset project: churn features with temporal validation and online mock store.
- Experiment: compare random and temporal split performance to expose leakage.
- Debugging: find timezone/default mismatch between offline and online paths.
- Extension: build an offline-online parity test harness.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Mini feature store	Registry, Parquet history, Redis latest lookup	Python, DuckDB, Redis	Strong ML-platform signal

Project	What it does	Stack/data	Resume value
Fraud feature service	Streaming velocity plus point-in-time backfill	Kafka, Spark/Flink	Low-latency ML systems
Feature parity lab	Compares reference/offline/online implementations	pytest/Python/SQL	Correctness ownership

19. Quick Revision

- Key idea: feature meaning includes entity, cutoff, window, source, default, and version.
- Formula: aggregate events in $(t - W, t]$.
- Metrics: freshness, coverage, parity, drift, incremental lift, fetch latency.
- Trap: "latest" joins and fitted preprocessing before split.
- Interview one-liner: "A feature is not just a column; it is a time-aware, versioned computation with serving semantics."

20. Final Cheat Sheet

Item	Summary
Input/output	Timestamped raw facts + entity/cutoff → versioned feature vectors
Main steps	define → validate → compute → PIT join → materialize/fetch → monitor
Key tuning	windows, TTL/freshness, defaults, materialization cadence
Metrics	parity, freshness, coverage, drift, latency, model lift
Pros/cons	reuse/consistency / state, versioning, leakage complexity
Best uses	production training and online/batch inference

Streaming ML

1. Overview

Streaming ML applies models and/or updates features and models continuously as events arrive. Common production systems do real-time inference with a fixed deployed model, update stateful features online, and retrain models periodically in batch. True online learning updates parameters per event or micro-batch and is appropriate only when rapid adaptation, reliable labels, and safe deployment justify the added risk.

2. Intuition

Batch ML studies yesterday's newspaper; streaming ML watches a live scoreboard. The scoreboard may receive late corrections, events out of order, and outcomes much later than predictions, so time and state are first-class.

3. Prerequisites

Supervised ML, inference, Kafka, event vs processing time, windows/watermarks, state stores, feature pipelines, concept drift, deployment and monitoring.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Real-time inference	Score each event/request using deployed model.	Fraud decision.	Latency budget
Online feature	Stateful value updated from events.	Purchases in 5 min.	Exactly-once/idempotency
Online learning	Incrementally update model parameters.	CTR logistic model.	When not to use it
Event time/watermark	Business time and bounded completeness estimate.	10-min allowed lateness.	Late-event handling
Stateful operator	Stores per-key/window information.	Running count.	State growth/TTL
Label join	Connect earlier prediction/example to delayed outcome.	Chargeback after weeks.	Attribution/leakage
Drift	$P(X)$ or $\$P(Y$	$X)\$$ changes.	New fraud strategy.
Shadow/canary	Evaluate new model without/all traffic.	5% rollout.	Safe deployment

5. Algorithm / Working Process

1. Ingest keyed, schema-versioned events and deduplicate by event ID.
2. Assign event timestamps and update bounded per-entity/window state.
3. Join request context with fresh online features.
4. Run versioned preprocessing and model inference within latency budget.
5. Emit prediction, model/feature version, timestamp, and explanation metadata.
6. Later ingest outcomes and join them to prediction/example IDs.
7. Compute delayed performance/drift; periodically retrain or cautiously update online.
8. Deploy through shadow/canary, monitor, and roll back on guardrail failure.

6. Mathematical Foundation

Online gradient descent updates

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \ell(f_{\theta_t}(x_t), y_t).$$

For logistic regression, $p_t = \sigma(\theta^T x_t)$ and cross-entropy is

$$\ell_t = -y_t \log p_t - (1 - y_t) \log(1 - p_t).$$

Exponentially decayed statistics use $s_t = \alpha x_t + (1 - \alpha)s_{t-1}$. Prequential evaluation predicts first, then scores once y_t arrives, avoiding training on the same example before evaluation. Regret compares cumulative online loss with the best fixed comparator: $R_T = \sum_t \ell_t(\theta_t) - \min_{\theta} \sum_t \ell_t(\theta)$.

7. Practical Implementation

```
import numpy as np

class OnlineLogisticRegression:
    def __init__(self, n_features: int, learning_rate: float = 0.05):
        self.w = np.zeros(n_features, dtype=float)
        self.lr = learning_rate

    def predict_proba(self, x: np.ndarray) -> float:
        z = float(np.clip(self.w @ x, -35, 35))
        return 1.0 / (1.0 + np.exp(-z))

    def learn_one(self, x: np.ndarray, y: int) -> float:
        p = self.predict_proba(x)          # prequential prediction
        self.w -= self.lr * (p - y) * x
        return p

model = OnlineLogisticRegression(3)
stream = [
    ("e1", np.array([1.0, 0.2, 0.0]), 1),
    ("e2", np.array([1.0, -0.5, 1.0]), 0),
]
seen, losses = set(), []
for event_id, x, y in stream:
    if event_id in seen:                    # idempotent replay guard for this demo
        continue
    seen.add(event_id)
    p = model.learn_one(x, y)
    losses.append(-(y * np.log(p + 1e-12) + (1-y) * np.log(1-p + 1e-12)))
assert np.isfinite(losses).all()
```

8. Code Explanation

The model computes probability before using the label, enabling honest prequential loss. The logistic gradient is $(p-y)x$. Clipping prevents exponential overflow. The event-ID set illustrates deduplication but is unbounded and in-memory; production uses durable keyed state/TTL and checkpoints. Feature scaling, model snapshots, delayed labels, and safe rollback are also required.

9. Training / Evaluation

For fixed-model streaming inference, train offline with temporal splits and evaluate online after labels mature. Monitor precision/recall at operating threshold, PR-AUC for imbalance, calibration, business cost, segment metrics, latency percentiles, availability, feature freshness, and label coverage/delay. Online learners use predict-then-learn evaluation, rolling metrics, drift alarms, learning-rate/regularization controls, champion-challenger comparisons, and frozen rollback snapshots.

10. Complexity and Cost

Linear-model inference/update is $O(d)$ time and $O(d)$ model memory per event. Window features require state proportional to active keys/events or compact summaries. Neural inference cost depends on model FLOPs and batching; strict latency reduces batch efficiency. Network/store calls often dominate. GPU helps high-throughput neural inference but not necessarily low-volume per-event scoring.

11. Common Use Cases

Fraud prevention, ad/recommendation ranking, anomaly detection, dynamic pricing signals, content moderation, predictive maintenance, cybersecurity, and real-time personalization.

12. Common Mistakes

- Calling real-time inference “online learning.”
- Updating a production model immediately on noisy, delayed, or attacker-controlled labels.
- Evaluating after learning on the same example.
- Ignoring out-of-order events and duplicate replays.
- Unbounded per-key/window state.
- Monitoring only input drift without outcome coverage/performance.
- Random split that overstates future deployment accuracy.
- No model snapshot, canary, or rollback.

13. Edge Cases / Limitations

Labels may arrive weeks later, be censored, or reflect interventions made by the model. Feedback loops bias future data. Rare keys create cold starts; hot keys create state skew. Watermarks may discard very late but valid corrections. Online adaptation can catastrophically forget or amplify attacks. When action latency is not truly valuable, micro-batch/batch is safer.

14. Variations

- Stream inference with fixed model: most common and placement-important.
- Streaming features + batch retraining: practical default for many production systems.
- Incremental classical learning: SGD, passive-aggressive, naive Bayes, trees designed for streams.
- Micro-batch training/inference: trades seconds of latency for efficiency.
- Continual learning: combats forgetting across tasks/distributions; research-heavy.
- Contextual bandits: learn from action-dependent feedback; important for recommendation research.

15. Related Topics

Kafka transports events, feature pipelines manage online/offline state, Spark/Flink execute stateful windows, Airflow handles periodic retraining, and data-quality/monitoring distinguishes pipeline faults from genuine drift.

16. Interview Questions

1. **Streaming inference vs online learning?** Streaming inference scores continuously with fixed parameters; online learning updates parameters as data/labels arrive.
2. **Event time vs processing time?** When an event occurred versus when the system handled it.

3. **What is a watermark?** A progress estimate used to close windows and bound state while accepting defined lateness.
4. **How do you evaluate an online learner?** Predict first, join delayed label later, then update; report rolling/segment metrics.
5. **How do you handle delayed labels?** Persist prediction/example IDs and versions, join outcomes later, and distinguish mature from pending cohorts.
6. **How do you detect drift?** Monitor inputs/features/predictions plus delayed supervised performance, segmented and seasonally baselined.
7. **Should drift trigger automatic retraining?** Not blindly; validate data health, outcome maturity, candidate performance, and deployment guardrails.
8. **What state must be checkpointed?** Offsets, window/feature state, dedup state, model/version, and transactional sink progress as applicable.
9. **How do you bound state?** Watermarks, TTL, window eviction, approximate summaries, and inactive-key cleanup.
10. **What is a feedback loop?** Model actions change observed data/labels, biasing future training and evaluation.

17. Practice Tasks

- Code: implement predict-then-learn logistic regression and rolling log loss.
- Dataset project: replay fraud/CTR events in event-time order.
- Experiment: inject abrupt/gradual drift and compare adaptation rates.
- Debugging: find duplicate feature updates after a consumer restart.
- Extension: add delayed-label store, shadow model, and canary guardrails.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Live fraud scorer	Stateful features, fixed model, feedback metrics	Kafka, FastAPI/Flink	End-to-end streaming ML
Drift simulator	Replays changing distributions and online learners	Python, River/NumPy	Research/interview depth
Streaming recommender	Updates user profile and ranks candidates	Kafka, Redis, PyTorch	Real-time AI system design

19. Quick Revision

- Key idea: continuous decisions require event-time state, delayed-label evaluation, and safe versions.
- Formula: $\theta_{t+1} = \theta_t - \eta_t \nabla \ell_t$.
- Metrics: p95/p99 latency, lag, freshness, mature-cohort PR-AUC/calibration, regret.
- Trap: learn-then-test, unbounded state, feedback loops.
- Interview one-liner: "Most streaming ML systems stream features and inference but retrain models safely in batch."

20. Final Cheat Sheet

Item	Summary
Input/output	Ordered/late events + model/state → decisions, updated state, logs
Main steps	ingest → dedup/window → fetch → predict → log → join label → evaluate/update
Key tuning	watermark, TTL, learning rate, threshold, batch size
Metrics	latency, lag, freshness, delayed performance, calibration
Pros/cons	fast/adaptive / state, labels, safety, cost complexity
Best uses	high-value low-latency decisions with changing behavior

Lakehouse Architecture

1. Overview

A lakehouse combines low-cost, open object-store files with warehouse-like table management: ACID transactions, schema enforcement/evolution, snapshots, concurrent reads/writes, governance, and SQL performance features. Table formats such as Delta Lake, Apache Iceberg, and Apache Hudi store metadata and transaction history around data files, commonly Parquet. ML benefits from reproducible snapshots, scalable feature tables, mixed SQL/Spark access, and raw-to-curated history in one governed platform.

2. Intuition

A plain data lake is a shared folder of files; a warehouse is a managed library. A lakehouse adds a catalog, edition history, checkout rules, and atomic shelf updates to the shared folder while retaining open, inexpensive storage.

3. Prerequisites

Object storage, Parquet, partitions, SQL/Spark, transactions, optimistic concurrency, snapshots, schemas, compaction, ETL/CDC, and warehouse concepts.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Data file	Immutable columnar file containing rows.	Parquet partition files.	Why immutable?
Metadata/manifest/log	Tracks files belonging to each snapshot.	Iceberg manifests/Delta log.	Atomic table view
Snapshot/time travel	Query a consistent historical table version.	Reproduce training set.	Retention caveat
ACID commit	Atomically publish a new metadata state.	Replace partition.	Concurrent writers

Subtopic	Meaning / why	Example	Interview angle
Schema evolution	Safely add/rename/change fields under rules.	Add nullable feature.	Field IDs/compatibility
Partition evolution	Change physical layout without rewriting query semantics.	Day to month partitioning.	Hidden partitioning
Compaction	Rewrite many small files into fewer larger files.	Streaming sink cleanup.	Read/write amplification
Upsert/delete	Merge changes and tombstones.	Apply CDC.	Copy-on-write vs merge-on-read

5. Algorithm / Working Process

1. Writer reads the current table snapshot/metadata.
2. It creates new immutable data/delete files in object storage.
3. It validates schema, constraints, and conflicts.
4. It atomically commits new metadata referencing the desired file set.
5. Readers resolve a snapshot and scan only files referenced by it, using partition/column/statistics pruning.
6. Maintenance compacts files, expires old snapshots, and removes unreachable files after safety windows.
7. Catalog/governance exposes table identity, permissions, lineage, and discovery.

6. Mathematical Foundation

Let snapshot S_v reference live file set F_v . A commit creates

$$F_{v+1} = (F_v \setminus F_{remove}) \cup F_{add}$$

and publishes the metadata pointer atomically. Readers pinned to v remain consistent while new readers see $v + 1$. Optimistic concurrency succeeds if no conflicting update invalidated the writer's base assumptions; otherwise it retries/rebases. Write amplification is $WA = \text{bytes rewritten} / \text{bytes logically changed}$.

7. Practical Implementation

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("lakehouse-features").getOrCreate()

# SQL syntax shown in Delta style; Iceberg/Hudi syntax/configuration differs.
spark.sql("""
CREATE TABLE IF NOT EXISTS ml.user_features (
  user_id STRING,
  feature_date DATE,
  spend_30d DOUBLE,
  event_count_7d BIGINT
) USING DELTA
PARTITIONED BY (feature_date)
""")
```

```
spark.sql("""
MERGE INTO ml.user_features AS target
USING staged_features AS source
ON target.user_id = source.user_id
AND target.feature_date = source.feature_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")

duplicates = spark.sql("""
SELECT user_id, feature_date, COUNT(*) AS n
FROM ml.user_features
GROUP BY user_id, feature_date HAVING n > 1
""").count()
assert duplicates == 0
```

8. Code Explanation

The table declares schema and date partition. **MERGE** makes keyed correction/upsert semantics explicit, though source keys must already be deduplicated; multiple source matches may fail or behave ambiguously. Readers see a committed snapshot rather than partially written files. Production workflows store the resulting table snapshot/version with each training run.

9. Training / Evaluation

Lakehouse infrastructure is evaluated by commit/read latency, scan pruning, file count/size, metadata growth, compaction cost, conflict/retry rate, freshness, and storage/compute cost. For ML reproducibility, verify that snapshot plus code/model configuration rebuilds the exact cohort/features and that retained snapshots/files survive the required audit window.

10. Complexity and Cost

Read cost depends on manifest lookup plus selected files/columns. Metadata planning suffers with huge file/partition counts. Updates may rewrite whole files (copy-on-write) or add delta/delete files merged during reads (merge-on-read). Compaction consumes I/O but reduces future opens and merge overhead. Object storage is cheap; repeated full scans and maintenance compute are not.

11. Common Use Cases

Raw/bronze, validated/silver, curated/gold layers; CDC tables; offline feature stores; training snapshots; BI marts; streaming-to-batch unification; audit/history; and large model metadata/corpus manifests.

12. Common Mistakes

- Calling a folder of Parquet a lakehouse without transactional metadata.
- Partitioning by user/item ID and creating massive metadata.
- Streaming tiny files without compaction.
- Expiring snapshots/vacuuming files needed for model reproducibility.

- Assuming ACID automatically enforces domain constraints or unique keys.
- Concurrent writers updating overlapping partitions without conflict design.
- Mixing table-format versions/features unsupported by some readers.

13. Edge Cases / Limitations

Object-store consistency and commit coordination differ across catalogs/formats. Long-running readers may need old files while cleanup runs. Partition evolution and schema changes require engine compatibility. Row-level updates can be expensive. A lakehouse does not automatically provide millisecond online serving, complete governance, or good data modeling.

14. Variations

- Delta Lake: transaction log, broad Spark/platform adoption; important in many ML stacks.
- Apache Iceberg: engine-neutral tables, hidden partitioning/evolution, manifest architecture; high industry relevance.
- Apache Hudi: strong incremental/upsert and copy-on-write/merge-on-read choices; CDC-heavy use.
- Medallion architecture: bronze/silver/gold quality layers; useful organizational convention, not a mandatory physical design.
- Copy-on-write vs merge-on-read: optimize read simplicity versus update speed.

15. Related Topics

Parquet is the data-file layer; Spark/warehouses are compute engines; ETL and Kafka populate tables; data-quality checks gate snapshots; feature pipelines record snapshot IDs; distributed internals explain atomic commits, retries, and metadata scalability.

16. Interview Questions

1. **Lake vs warehouse vs lakehouse?** Lake stores flexible low-cost files; warehouse provides managed analytics; lakehouse adds table/transaction capabilities over open lake storage.
2. **Why is Parquet alone not a table?** Files lack a single atomic definition of membership, transaction history, concurrency, and evolution.
3. **How do snapshots work?** Metadata versions reference immutable file sets; readers pin a consistent version.
4. **How are atomic commits possible on object storage?** Writers create immutable files then atomically publish/update small metadata through a catalog/log protocol.
5. **What is compaction?** Rewriting small/delta files into larger optimized files without changing logical rows.
6. **Copy-on-write vs merge-on-read?** COW rewrites data on update for faster reads; MOR records deltas for faster writes and merges later/on read.
7. **How does time travel help ML?** A training run can reference the exact historical data snapshot used.
8. **What can break time travel?** Snapshot expiration or deletion/vacuum of referenced files.
9. **Does ACID guarantee no duplicate keys?** Not necessarily; uniqueness must be enforced by pipeline/table constraints where supported.
10. **How do you choose partitions?** Use common low/moderate-cardinality filters such as date, target useful file sizes, and rely on clustering/statistics for finer pruning.

17. Practice Tasks

- Code: create, merge, snapshot, and time-travel a small feature table.
- Dataset project: build bronze/silver/gold layers from CDC-like events.
- Experiment: benchmark file count and compaction effects.
- Debugging: reproduce missing historical snapshot after unsafe cleanup.
- Extension: handle schema and partition evolution across readers.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Reproducible ML lakehouse	Pins feature snapshots to model runs	Spark, Iceberg/Delta, MLflow	Strong MLOps/data platform story
CDC customer table	Applies updates/deletes and exposes history	Kafka, Hudi/Iceberg	Incremental-data depth
Small-file optimizer	Profiles and compacts partitions safely	Spark, table metadata	Performance operations

19. Quick Revision

- Key idea: immutable files plus transactional metadata produce governed, versioned tables.
- Formula: $F_{v+1} = (F_v - F_{remove}) \cup F_{add}$.
- Metrics: file count/size, scan bytes, conflicts, compaction, freshness.
- Trap: vacuuming reproducibility and uncontrolled small files.
- Interview one-liner: "A lakehouse turns object-store files into atomic snapshot tables consumable by multiple engines."

20. Final Cheat Sheet

Item	Summary
Input/output	Data/delete files + metadata commits → versioned table snapshots
Main steps	write immutable files → validate/conflict-check → commit metadata → maintain
Key tuning	file size, partitions/clustering, compaction, retention
Metrics	planning/read/commit latency, files, conflicts, scan/cost
Pros/cons	open, scalable, reproducible / metadata and maintenance complexity
Best uses	lake ETL, feature history, CDC, ML snapshots, mixed analytics

Distributed Data Processing Internals

1. Overview

Distributed data processing executes a logical computation across machines whose CPUs, memory, disks, and networks are independent and can fail. Core internals include partitioning, scheduling, shuffles, serialization, memory/spill, fault tolerance, replication, consistency, stragglers, and distributed aggregation. Understanding them explains Spark/Kafka/warehouse behavior and is a frequent differentiator in data/ML systems interviews.

2. Intuition

Splitting a book among translators is easy if chapters are independent. It becomes hard when every translator must build one alphabetized index: entries must move to the worker responsible for each letter, slow translators delay completion, and failed work must be reconstructed without duplicating the final publication.

3. Prerequisites

Big-O analysis, hashing/sorting, probability basics, operating systems, networks, serialization, databases, DAGs, joins/aggregations, and fault models.

4. Core Concepts

Subtopic	Meaning / why	Example	Interview angle
Partitioning	Map records to parallel shards.	<code>hash(key) mod P</code> .	Repartitioning/skew
Task/stage/job	Partition work, shuffle-delimited group, action-level computation.	Spark execution.	Critical path
Shuffle	Redistribute keyed records across network/disk.	group-by/join.	Why expensive?
Serialization	Encode objects for storage/network.	Arrow/Protobuf/Kryo.	CPU/compatibility tradeoff
Locality	Run compute near input to reduce network.	HDFS block locality.	Object-store differences
Spill	Move working state to disk when memory insufficient.	External sort.	Performance diagnosis
Lineage/checkpoint	Recompute from ancestors vs persist state.	Lost partition recovery.	Tradeoff
Delivery/commit	Coordinate retries with side effects.	Attempt-specific staged files.	Exactly-once effect
Replication	Multiple copies tolerate failures/read load.	Kafka RF=3.	Consistency/durability
Backpressure	Bound producers when downstream is slower.	Growing stream lag.	Stability

5. Algorithm / Working Process

For a typical distributed group-by:

1. Coordinator converts logical operators into stages and partition tasks.
2. Workers read splits and apply local filter/projection/map.
3. A combiner computes mergeable partial aggregates per key.
4. Partitioner assigns keys to reducers; map outputs are serialized, sorted/bucketed, and spilled if necessary.
5. Reducers fetch shuffle blocks, merge states, and write attempt-scoped outputs.
6. Coordinator retries lost tasks and commits only successful outputs atomically.
7. Metrics/heartbeats expose progress, skew, spill, and failure.

6. Mathematical Foundation

Hash partitioning is $p(k) = h(k) \bmod P$. Ideal task load is N/P , but stage time follows the maximum load. Distributed work and communication can be described as

$$T \approx \max_j (T_{read,j} + T_{cpu,j} + T_{network,j} + T_{spill,j}) + T_{coord}.$$

Amdahl's law bounds parallel speedup; Gustafson's law explains scaling problem size with workers. For independent machine failure probability q , simple r -replica data-loss probability is roughly q^r only under unrealistic independence. Consistent hashing maps nodes/keys to a ring so adding a node moves about $1/(n+1)$ of keys, subject to virtual-node balance.

Map-side aggregation is valid for associative, commutative merge state. Exactly computing average by partial state uses $(s, n) \oplus (s', n') = (s + s', n + n')$. At-least-once tasks require idempotent or transactional sinks because retries may repeat side effects.

7. Practical Implementation

```
from collections import defaultdict
from typing import Iterable

def map_partition(rows: Iterable[tuple[str, float]]) -> dict[str, tuple[float, int]]:
    """Local combiner: drastically reduces shuffle for repeated local keys."""
    partial = defaultdict(lambda: [0.0, 0])
    for key, value in rows:
        partial[key][0] += value
        partial[key][1] += 1
    return {key: (state[0], state[1]) for key, state in partial.items()}

def reduce_partials(parts: Iterable[dict[str, tuple[float, int]]]) -> dict[str, float]:
    merged = defaultdict(lambda: [0.0, 0])
    for part in parts:
        for key, (subtotal, count) in part.items():
            merged[key][0] += subtotal
            merged[key][1] += count
    return {key: total / count for key, (total, count) in merged.items()}
```

```
partials = [map_partition([("a", 1), ("a", 3)]), map_partition([("a", 5), ("b", 8)])]
assert reduce_partials(partials) == {"a": 3.0, "b": 8.0}
```

8. Code Explanation

Each mapper emits one `(sum, count)` state per local key instead of every original value. Reducers merge the fixed-size states and finalize the mean. This is the core combiner principle used by distributed engines. A production implementation uses stable binary serialization, partitioned network transfer, spillable maps, retry-safe outputs, and numerically stable summation when accuracy demands it.

9. Training / Evaluation

The system is configured rather than trained. Benchmark representative data size, record shape, key skew, and failure conditions. Measure stage/task latency distributions, CPU, I/O, network/shuffle, spill, serialization, GC, scheduler delay, retries, locality, and output correctness. For distributed ML training also measure accelerator utilization, communication-to-compute ratio, convergence equivalence, checkpoint overhead, and straggler sensitivity.

10. Complexity and Cost

- Embarrassingly parallel map: $O(N/P)$ ideal per worker.
- Shuffle: $O(N)$ bytes in the worst common case plus network/disk/serialization.
- Global sort: local $O((N/P) \log(N/P))$ plus range partition/shuffle and merge.
- Hash join: linear expected compute but build memory and shuffle may dominate.
- Driver/coordinator metadata commonly scales with tasks/files/partitions, creating a control-plane bottleneck.
- GPUs accelerate model kernels only if input pipelines and collective communication keep them fed.

11. Common Use Cases

Large ETL, search indexing, feature generation, graph processing, distributed SQL, stream aggregation, model training with data/model parallelism, embedding generation, and fault-tolerant event processing.

12. Common Mistakes

- Assuming linear speedup with workers.
- Measuring averages and missing p99 stragglers/skew.
- Choosing partition count without considering data size and scheduler/file overhead.
- Large centralized `collect`, metadata list, or single reducer.
- Non-associative combiner or average-of-averages.
- Retried tasks causing duplicate external effects.
- Overlooking serialization and network as dominant costs.
- Confusing replication with backup or exactly-once processing.

13. Edge Cases / Limitations

Correlated rack/zone failures violate independence assumptions. Network partitions create availability/consistency tradeoffs. Hot keys defeat hash balance; speculative execution can duplicate non-idempotent side effects. Floating-point results vary with reduction order. Distributed deadlocks, slow disks, GC pauses, clock skew, and metadata hotspots may mimic data skew. Some workloads fit one machine and should stay there.

14. Variations

- MapReduce: materialized map/shuffle/reduce stages; foundational interview model.
- DAG engines: pipeline multiple operators and cache/recompute lineage; Spark/Tez.
- BSP/graph processing: synchronized supersteps; graph algorithms.
- Data parallel training: replicate model, shard samples, all-reduce gradients.
- Model/tensor/pipeline parallelism: shard large networks across devices; research/LLM systems relevance.
- Actor systems: stateful communicating processes; useful for serving/simulation.

15. Related Topics

Spark exposes DAG/shuffle internals; Kafka exposes partition/replication/offset semantics; warehouses hide similar distributed scans and exchanges; lakehouses solve file-level atomic publication; streaming ML adds state/checkpoints/backpressure; distributed DL adds collectives.

16. Interview Questions

1. **Why is shuffle expensive?** It adds serialization, network transfer, sorting/buffering, disk spill, synchronization, and new failure boundaries.
2. **What causes a stage boundary?** A wide dependency requiring data redistribution/materialization.
3. **How do you choose partition count?** Enough for parallelism and balanced retry units, but large enough that task/file overhead is small; benchmark real data.
4. **How do you handle skew?** Measure key/partition tails, pre-aggregate, split/salt hot keys, isolate them, or use skew-aware join strategies.
5. **Lineage vs checkpoint?** Lineage saves storage and recomputes lost data; checkpoints bound recovery time/dependency depth at I/O cost.
6. **What is backpressure?** Feedback that reduces intake or buffers safely when downstream throughput is lower than arrival rate.
7. **Why can retries duplicate output?** A worker may finish an external write but fail before the coordinator records success; commit protocols/idempotency are needed.
8. **What is data locality?** Scheduling computation close to stored data to reduce network I/O; less direct with remote object stores.
9. **Why does more memory help only sometimes?** It reduces spill/cache misses, but cannot fix skew, network, serialization, source limits, or serial work.
10. **How does distributed DL synchronize gradients?** Data-parallel workers compute local gradients then aggregate, often through ring/tree all-reduce, before consistent parameter updates.

17. Practice Tasks

- Code: implement mergeable mean/variance partial states.
- Dataset project: run a distributed group-by and profile shuffle/skew.

- Experiment: vary partition count and plot throughput/task overhead.
- Debugging: distinguish GC, spill, and hot-key stragglers from task metrics.
- Extension: simulate worker failure and idempotent staged commit.

18. Project Ideas

Project	What it does	Stack/data	Resume value
Mini MapReduce	Map, partition, shuffle, combine, reduce, retry locally	Python stdlib	Demonstrates internals clearly
Distributed profiler	Reports skew, spill, file/task size and fixes	Spark	Production diagnosis
All-reduce benchmark	Compares single vs multi-process model training	PyTorch Distributed	Deep-learning systems depth

19. Quick Revision

- Key idea: partition local work; pay network/state cost only where global coordination is required.
- Formula: stage time is dominated by the slowest partition; Amdahl bounds speedup.
- Metrics: p50/p99 task time, shuffle, spill, skew, serialization, retries, utilization.
- Trap: linear-speedup assumptions and retry-unsafe effects.
- Interview one-liner: "Distributed performance is usually a problem of data movement, imbalance, and coordination—not raw arithmetic."

20. Final Cheat Sheet

Item	Summary
Input/output	Partitioned data + DAG → committed distributed result/state
Main steps	split → schedule → local compute → shuffle/merge → retry → commit
Key tuning	partitioning, parallelism, memory/spill, serialization, checkpoints
Metrics	critical path, skew, shuffle, spill, GC, retry, network
Pros/cons	scale/fault tolerance / coordination, cost, nondeterminism
Best uses	datasets/workloads beyond one machine or needing distributed availability

Cross-Topic System Design Checklist

Use this sequence in placement interviews when asked to design an ML data platform:

1. State the product decision, latency, volume, correctness, retention, privacy, and availability requirements.
2. Declare entities, event IDs/timestamps, table grain, schemas, and ownership.
3. Choose batch unless the required latency proves streaming is necessary.

4. Use Kafka for replayable events, object storage + Parquet/lakehouse for durable history, and a warehouse/Spark for analytical computation.
5. Define cutoff-aware labels/features and training-only fitted preprocessing.
6. Make writes idempotent and publication atomic; specify retry, checkpoint, late-data, deduplication, and backfill behavior.
7. Add quality gates at trust boundaries and monitor freshness, volume, distributions, lineage, lag, cost, and downstream model metrics.
8. Identify scaling bottlenecks: hot keys, shuffles, small files, state growth, online-store calls, GPU starvation, and critical-path stages.
9. Version schema, code, data snapshot, features, preprocessing, and model so a prediction or training run can be reproduced.
10. Explain failure behavior explicitly: what can be lost, duplicated, delayed, rolled back, replayed, or recomputed.